

Pontifícia Universidade Católica de Campinas

Estruturas e Recuperação de Dados II



LISTAS



Prof. Me. Darcy H.F.R. Moreira



ESTRUTURAS LINEARES DINÂMICAS

- **Não** tem tamanho fixo – crescem e diminuem conforme a necessidade
- Utilizam **ponteiros** → A ligação entre os elementos não é contígua na memória, e sim feita por “**nós**” conectados entre si.
- São **flexíveis** → Permitem **inserções** e **remoções** sem precisar deslocar grandes blocos de memória

ESTRUTURAS LINEARES DINÂMICAS

- **Lista Encadeada (Linked List)** — Consiste em uma **sequência de nós**, onde cada nó contém um dado e um ponteiro para o próximo nó.
 - Permitindo **inserções** e **remoções** eficientes em **qualquer** posição

ESTRUTURAS LINEARES DINÂMICAS

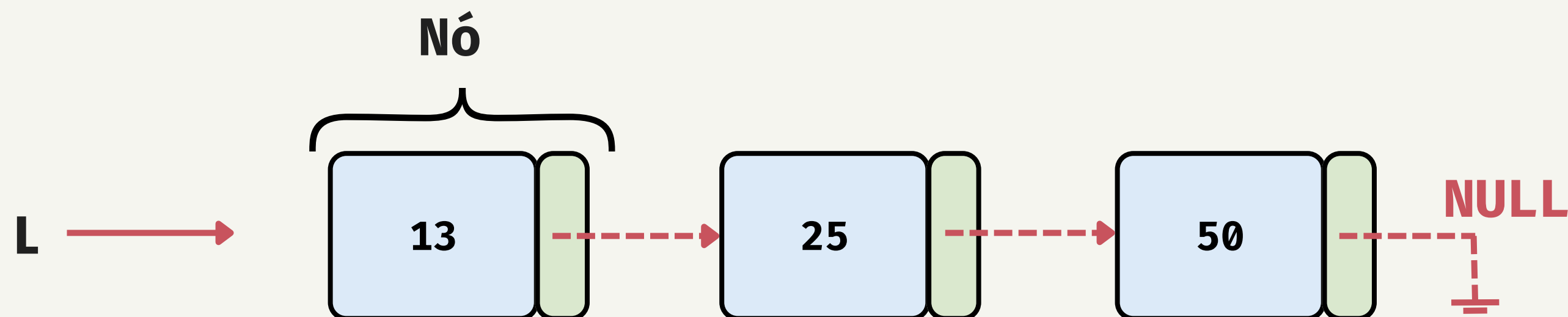
- **Lista Encadeada (Linked List)** — Consiste em uma **sequência de nós**, onde cada nó contém um dado e um ponteiro para o próximo nó.
 - Permitindo **inserções** e **remoções** eficientes em **qualquer** posição
- **Pilha (Stack)** — Baseada no princípio **LIFO** (Last In, First Out). Elementos são **inseridos** e **removidos** do **topo**.

ESTRUTURAS LINEARES DINÂMICAS

- **Lista Encadeada (Linked List)** — Consiste em uma **sequência de nós**, onde cada nó contém um dado e um ponteiro para o próximo nó.
 - Permitindo **inserções** e **remoções** eficientes em **qualquer** posição
- **Pilha (Stack)** — Baseada no princípio **LIFO** (Last In, First Out). Elementos são **inseridos** e **removidos** do **topo**.
- **Fila (Queue)** — Estrutura de dados baseada no princípio **FIFO** (First In, First Out). Elementos são **inseridos** no **final** e **removidos** do **início**.
 - Ideal para processar itens na **ordem** em que **chegam**.

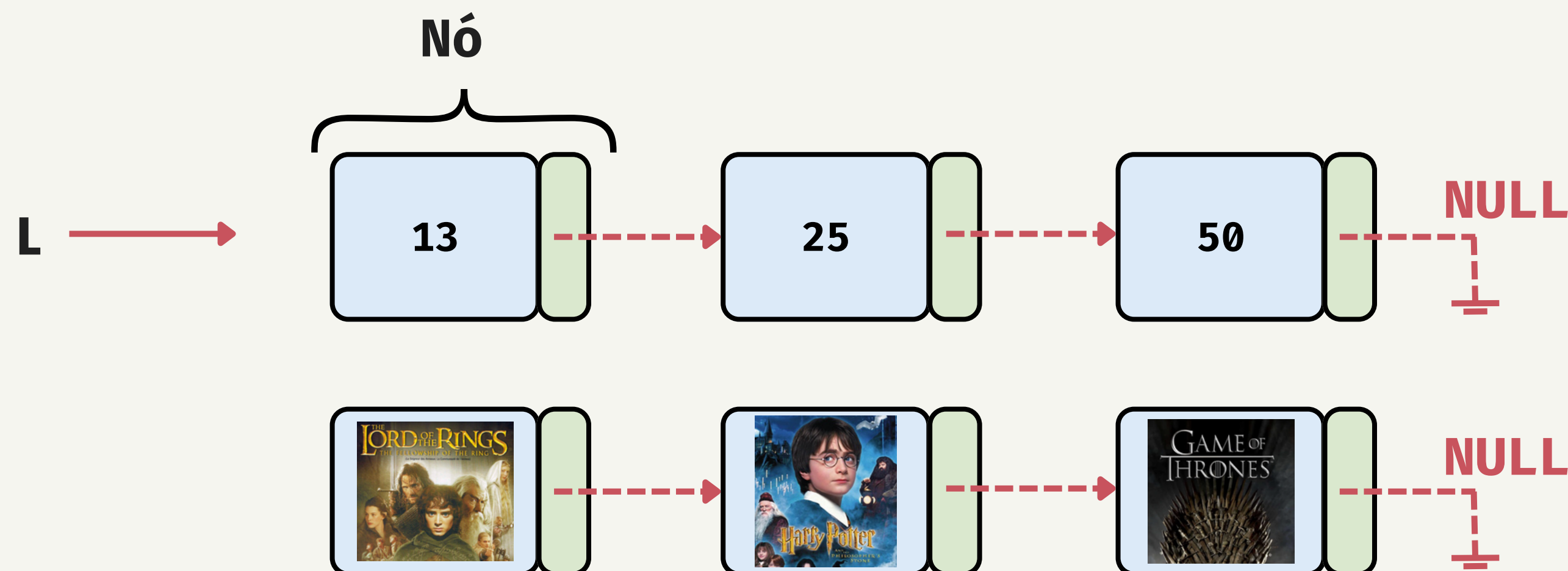
LISTA ENCADEADA/LIGADA

- Sequência de **elementos/objetos** na memória.
- Os **elementos** (nós da lista) são armazenados em posições **quaisquer** da memória e são ligados por **ponteiros**.
 - elementos **consecutivos** da lista **não** ficam necessariamente em **posições consecutivas na memória**.



LISTA ENCADEADA/LIGADA

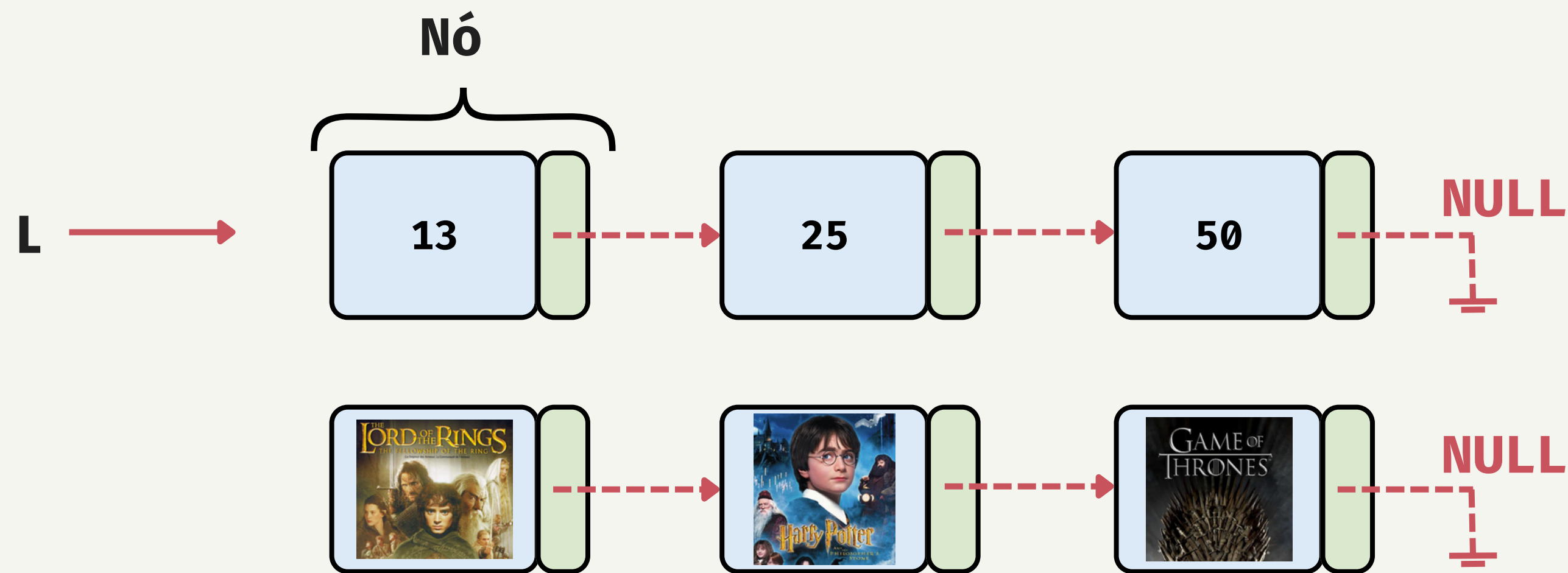
- Sequência de **elementos/objetos** na memória.
- Os **elementos** (nós da lista) são armazenados em posições **quaisquer** da memória e são ligados por **ponteiros**.
 - elementos **consecutivos** da lista **não** ficam necessariamente em **posições consecutivas na memória**.



LISTA ENCADEADA/LIGADA



- Alocação **sob demanda**
- **Inserção e remoção** em qualquer lugar (mais **flexível**)



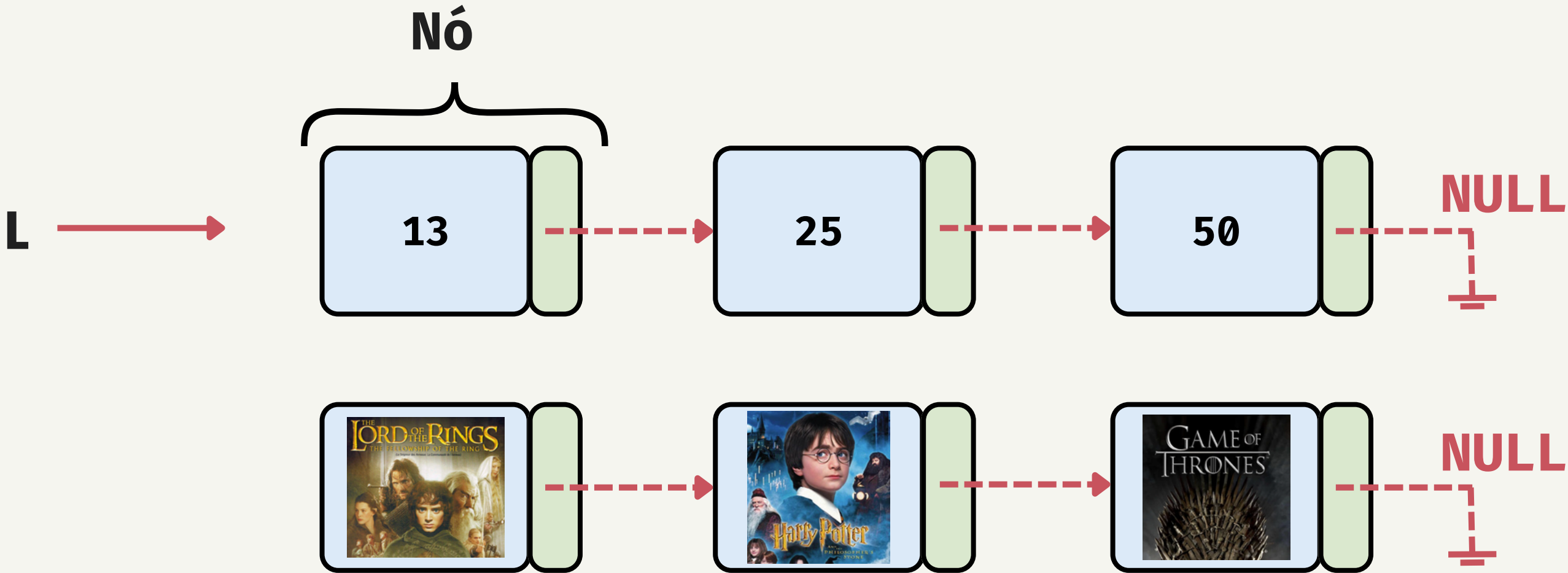
LISTA ENCADEADA/LIGADA



- Alocação **sob demanda**
- Inserção** e **remoção** em qualquer lugar (mais **flexível**)

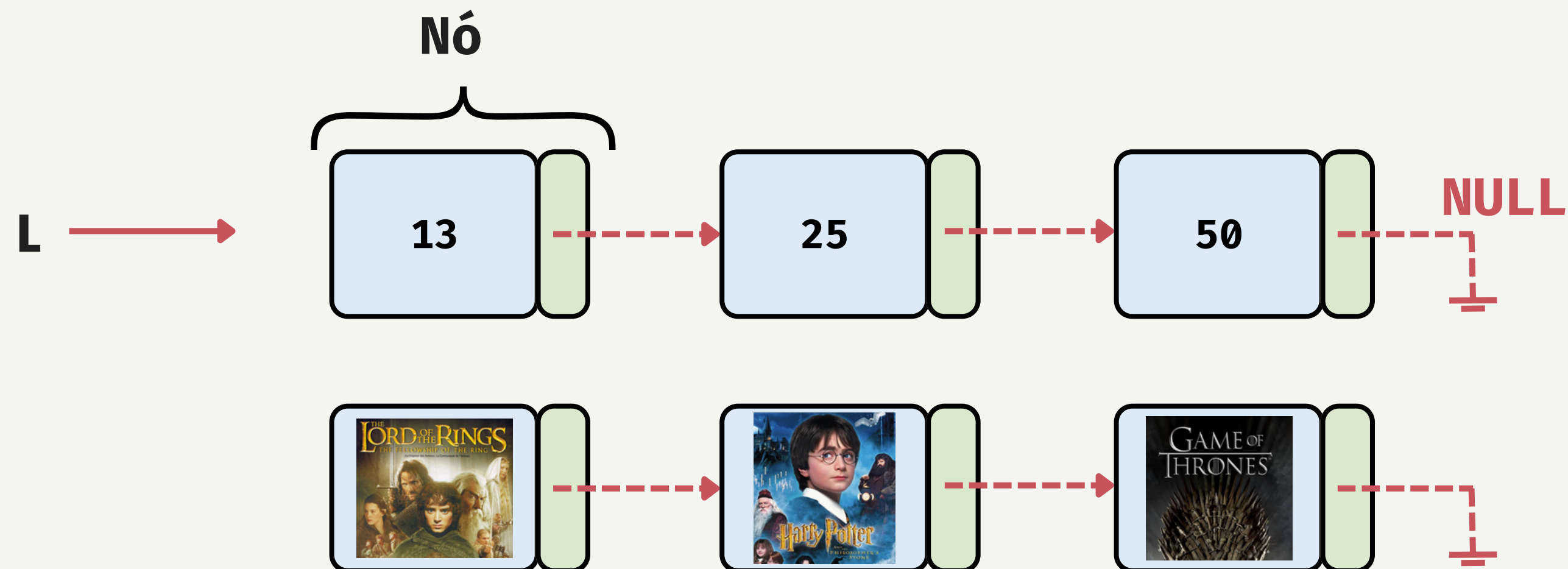


- Busca** de elementos e **inserção** e **remoção** pelo elemento é mais **complexa** (alocação não é linear);



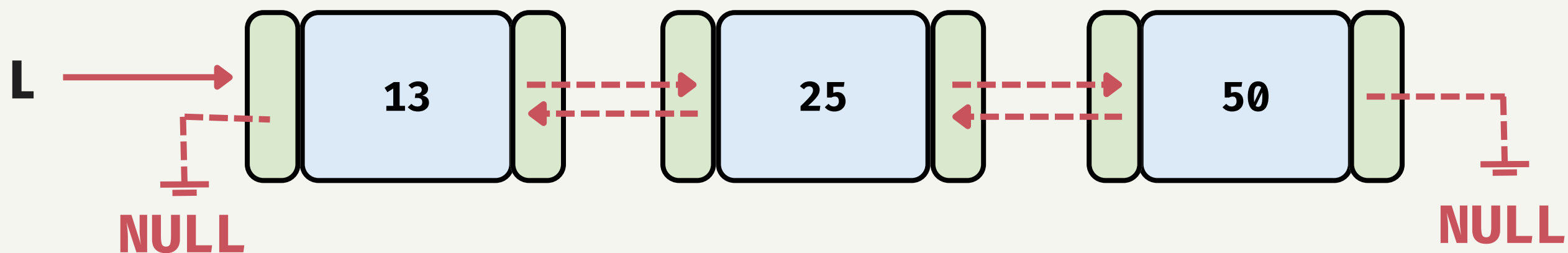
LISTA ENCADEADA/LIGADA

- **Lista encadeada simples:** cada nó contém **um elemento** de determinado tipo e **um ponteiro** para o próximo nó.
- No último nó, este ponteiro aponta para **NULL**.



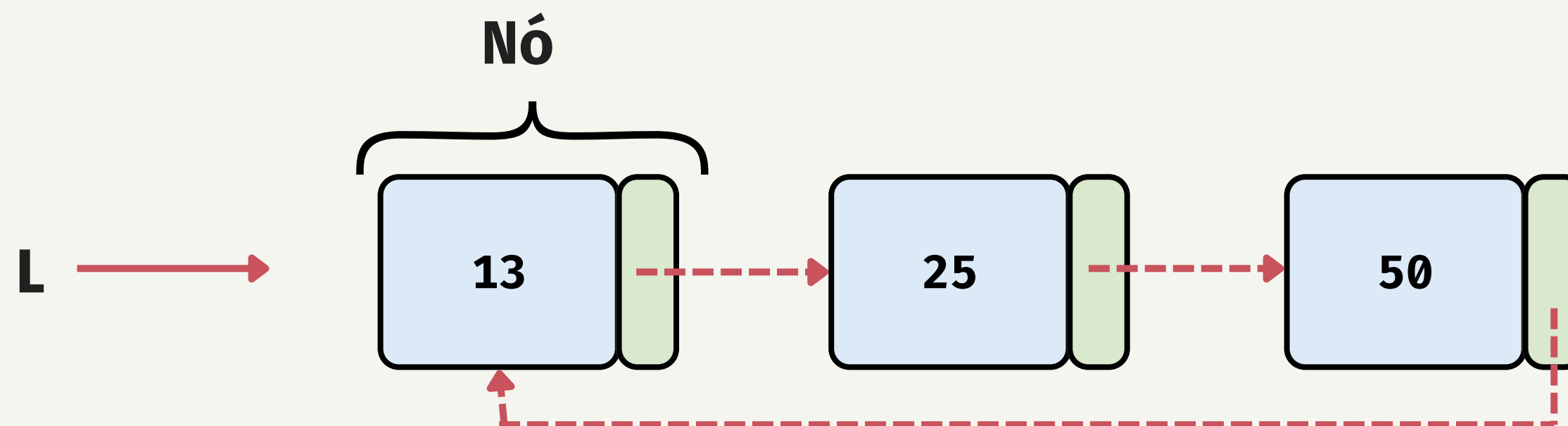
LISTA ENCADEADA/LIGADA

- **Lista duplamente encadeada:** cada nó aponta para o próximo e o nó anterior (**dois ponteiros**).



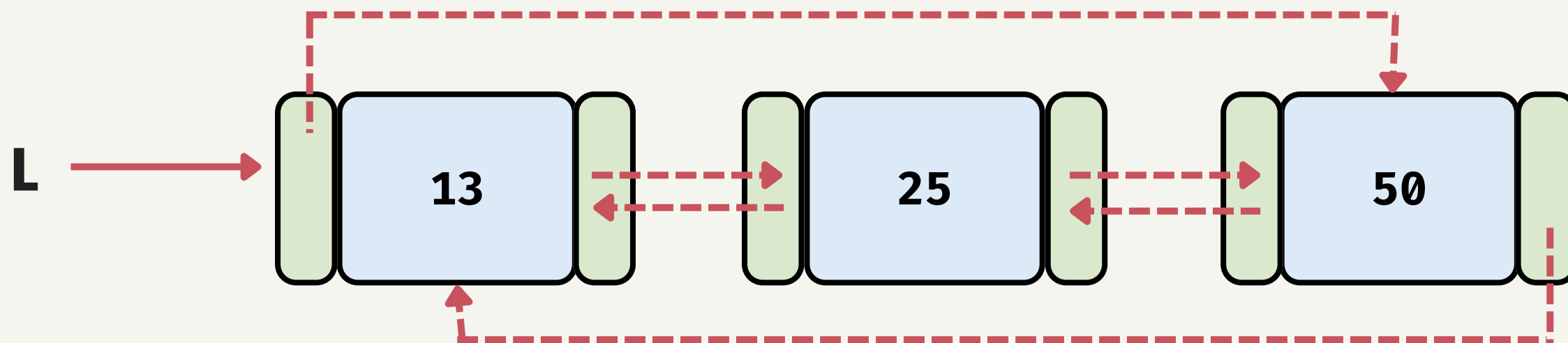
LISTA ENCADEADA/LIGADA

- **Lista circular:** o **próximo** nó do último elemento da lista aponta para o **primeiro**.



LISTA ENCADEADA/LIGADA

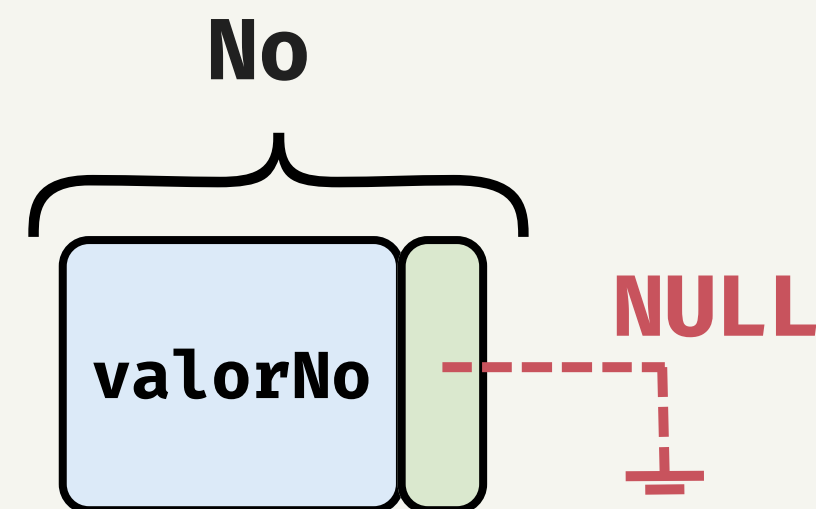
- **Lista circular duplamente encadeada:** cada nó aponta para o **próximo** e o nó **anterior**, o último nó aponta para o **primeiro** e o primeiro aponta para o **último**.



LISTA ENCADEADA SIMPLES

- Criando uma **Struct Nó**

```
typedef struct No {  
    int valorNo;  
    struct No* proxNo;  
} No;
```

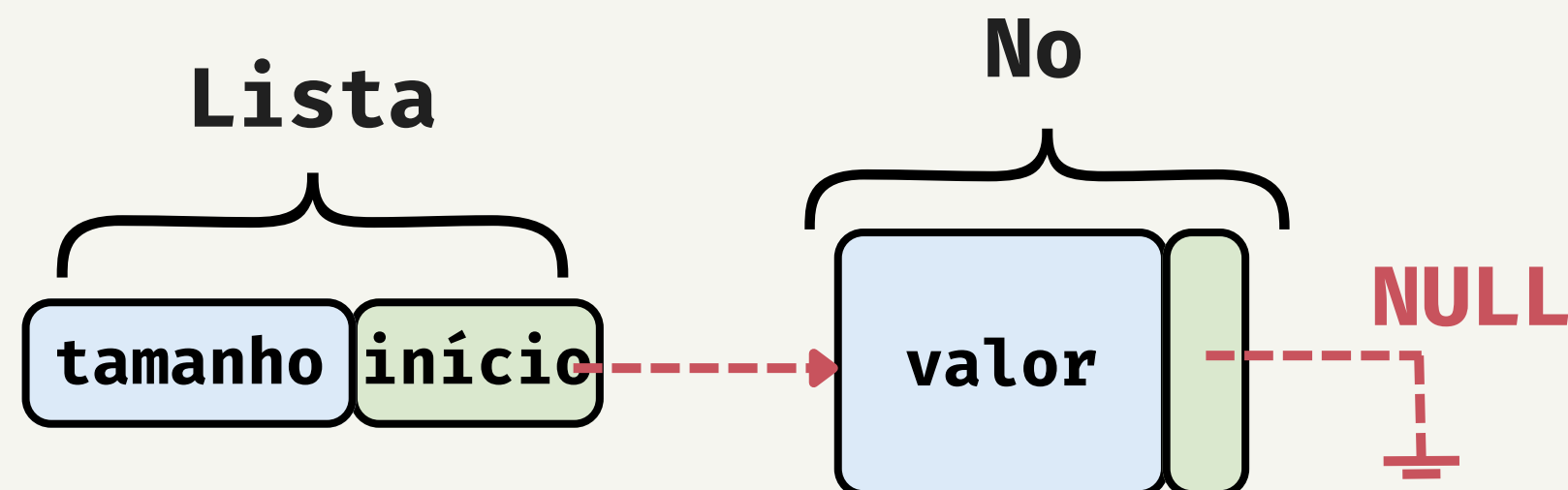


LISTA ENCADEADA SIMPLES

- Criando uma **Struct Nó**
- Criando uma **Struct Lista**
 - Acesso às informações principais da lista

```
typedef struct No {
    int valorNo;
    struct No* proxNo;
} No;
```

```
typedef struct {
    No* inicio;
    int tamanho;
} Lista;
```



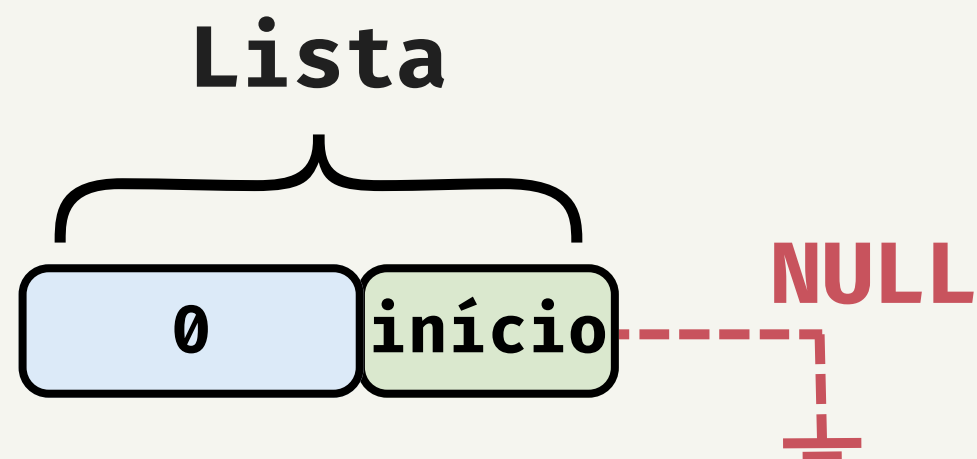
LISTA ENCADEADA SIMPLES

Função **criarLista()** – ainda não adicionei nenhum nó

```

Lista* criarLista() {
    Lista* lista = (Lista*)malloc(sizeof(Lista));

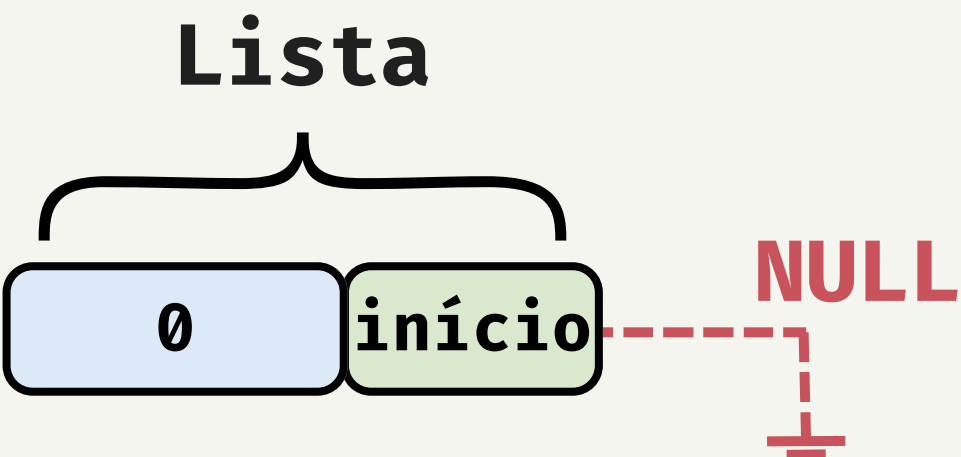
    if(lista){
        lista->inicio = NULL;
        lista->tamanho = 0;
    }
    return lista;
}
    
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

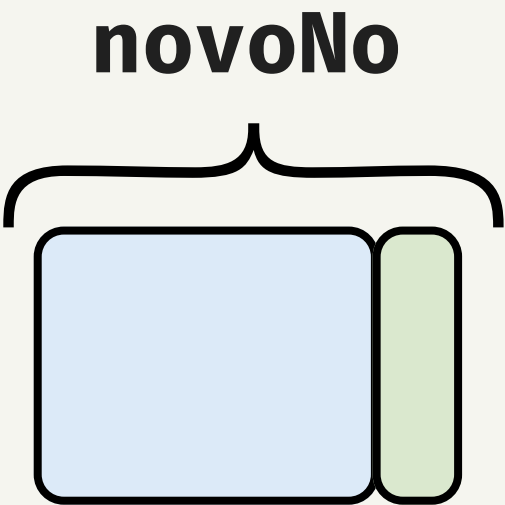
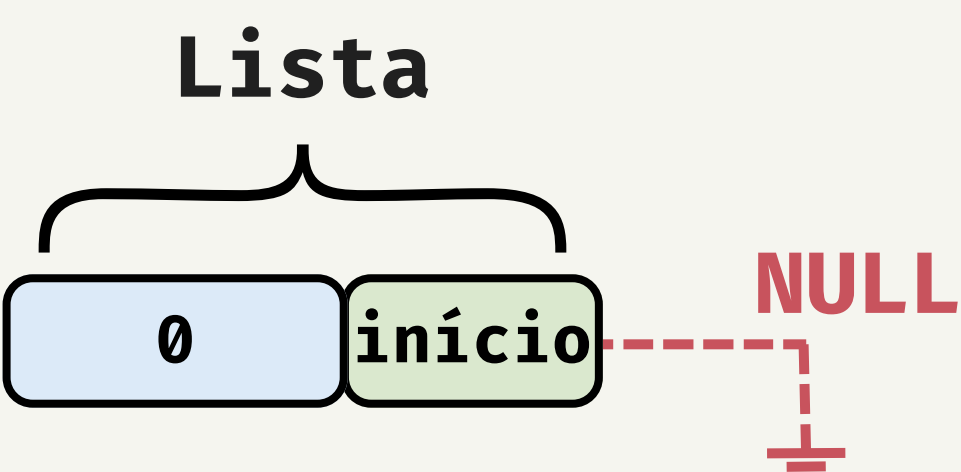
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

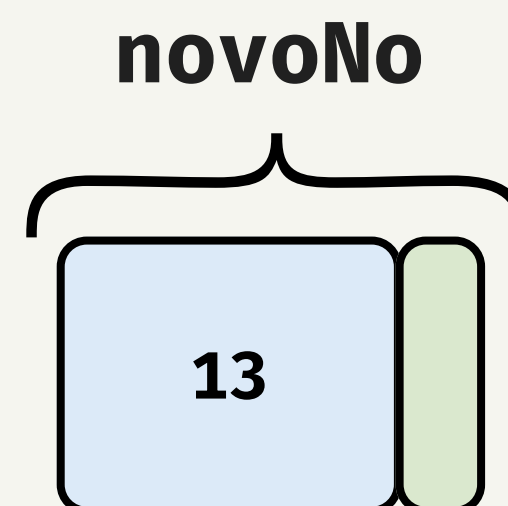
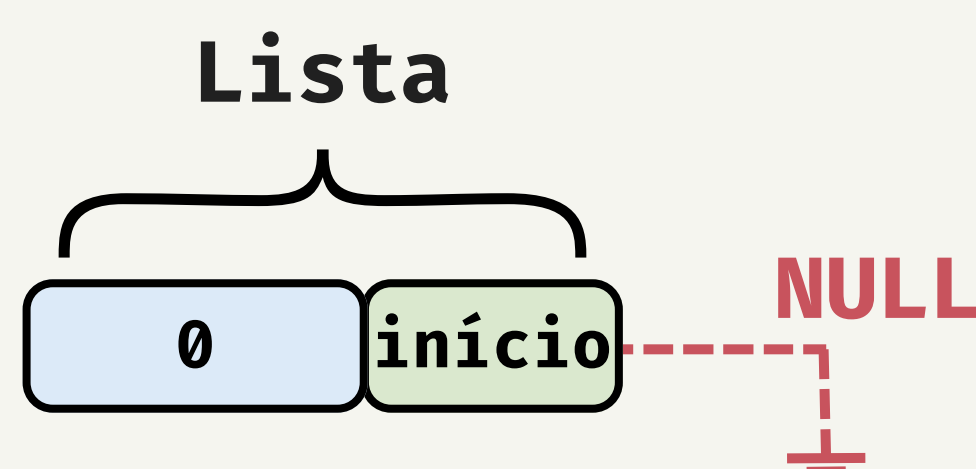
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

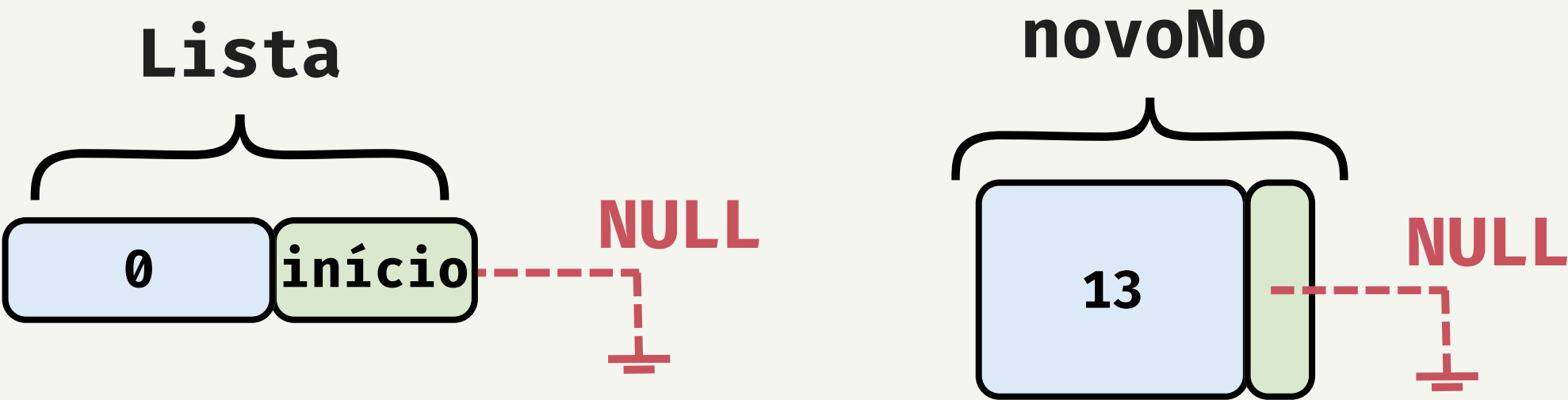
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

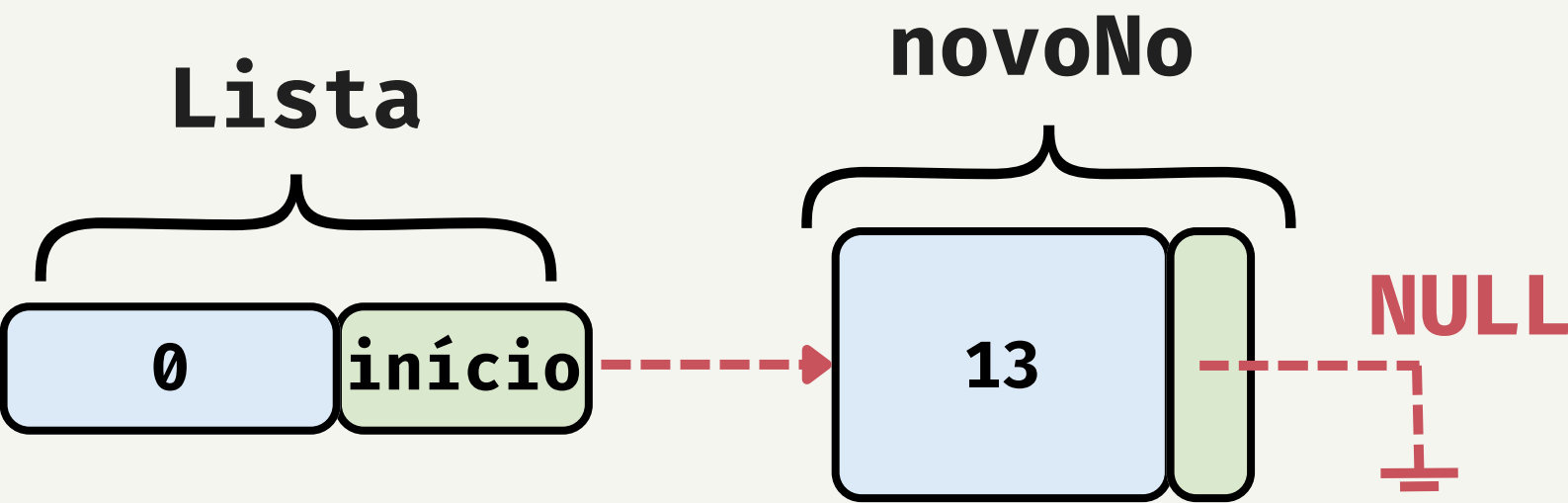
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

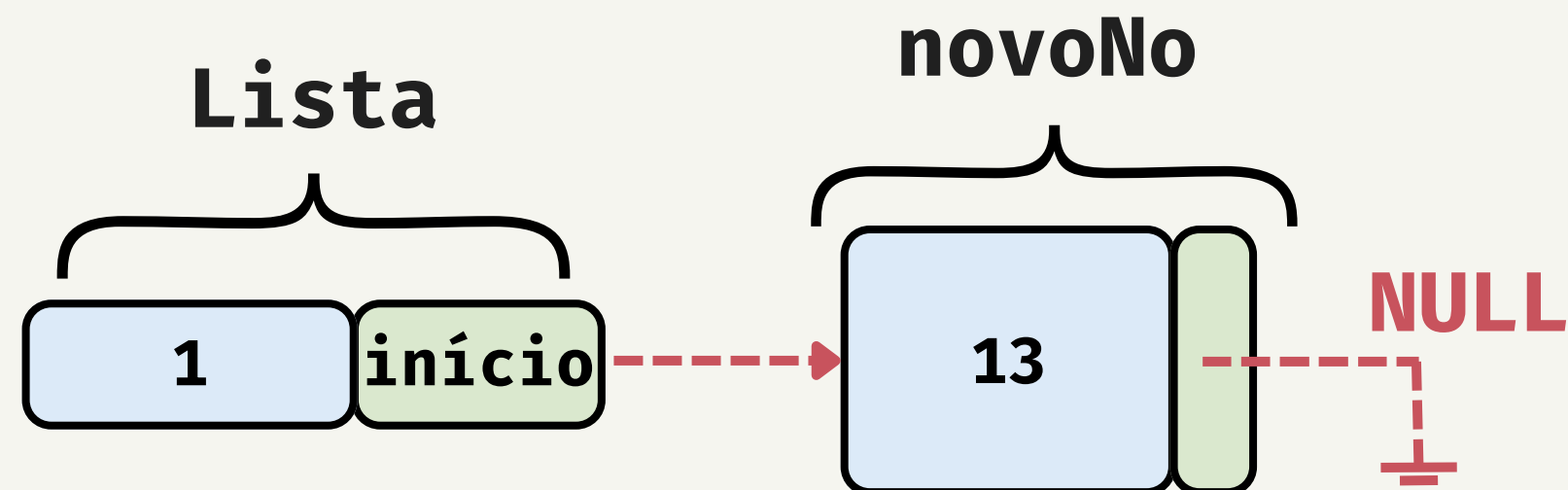
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

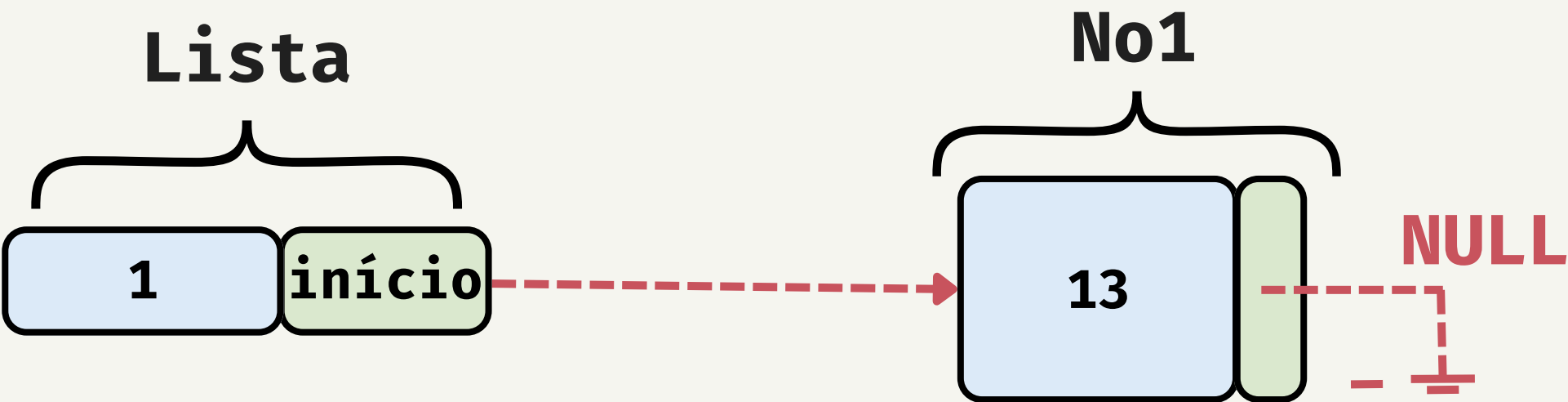
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```

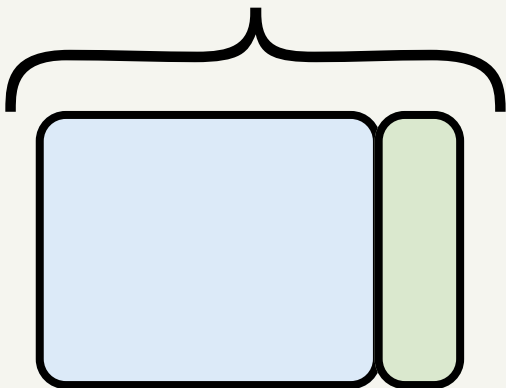


LISTA ENCADEADA SIMPLES

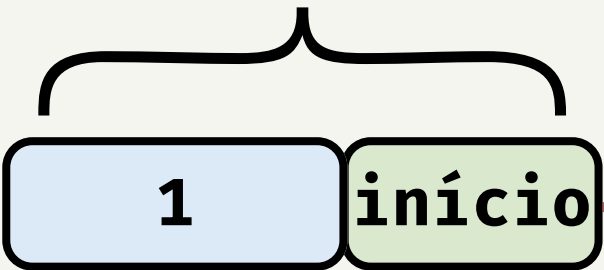
Função **inserirInicio()**

```
void inserirInicio(Lista* lista, int valor) {  
    No* novoNo = (No*)malloc(sizeof(No));  
    if(novoNo){  
        novoNo->valorNo = valor;  
        novoNo->proxNo = lista->inicio;  
        lista->inicio = novoNo;  
        lista->tamanho++;  
    }  
}
```

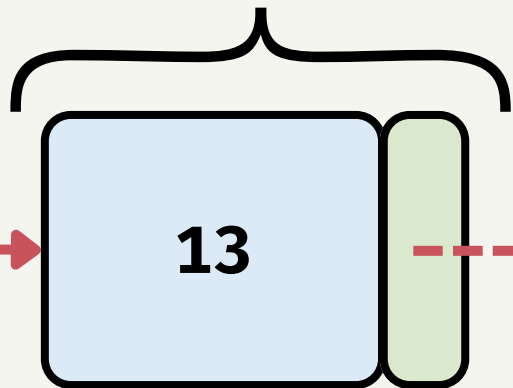
novoNo



Lista



No1



NULL

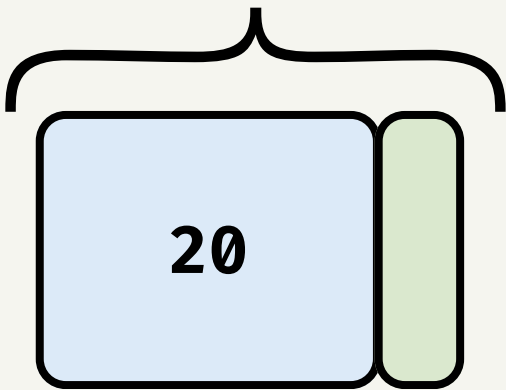


LISTA ENCADEADA SIMPLES

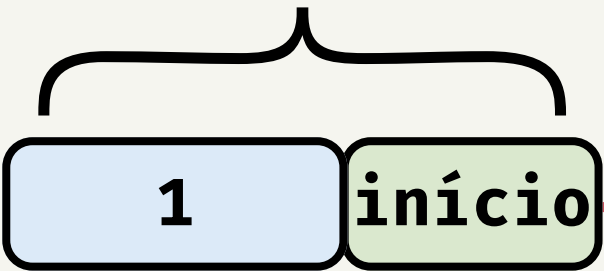
Função **inserirInicio()**

```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```

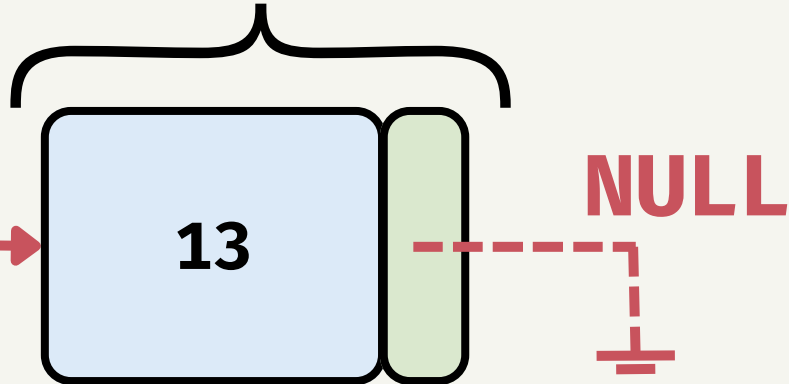
novoNo



Lista



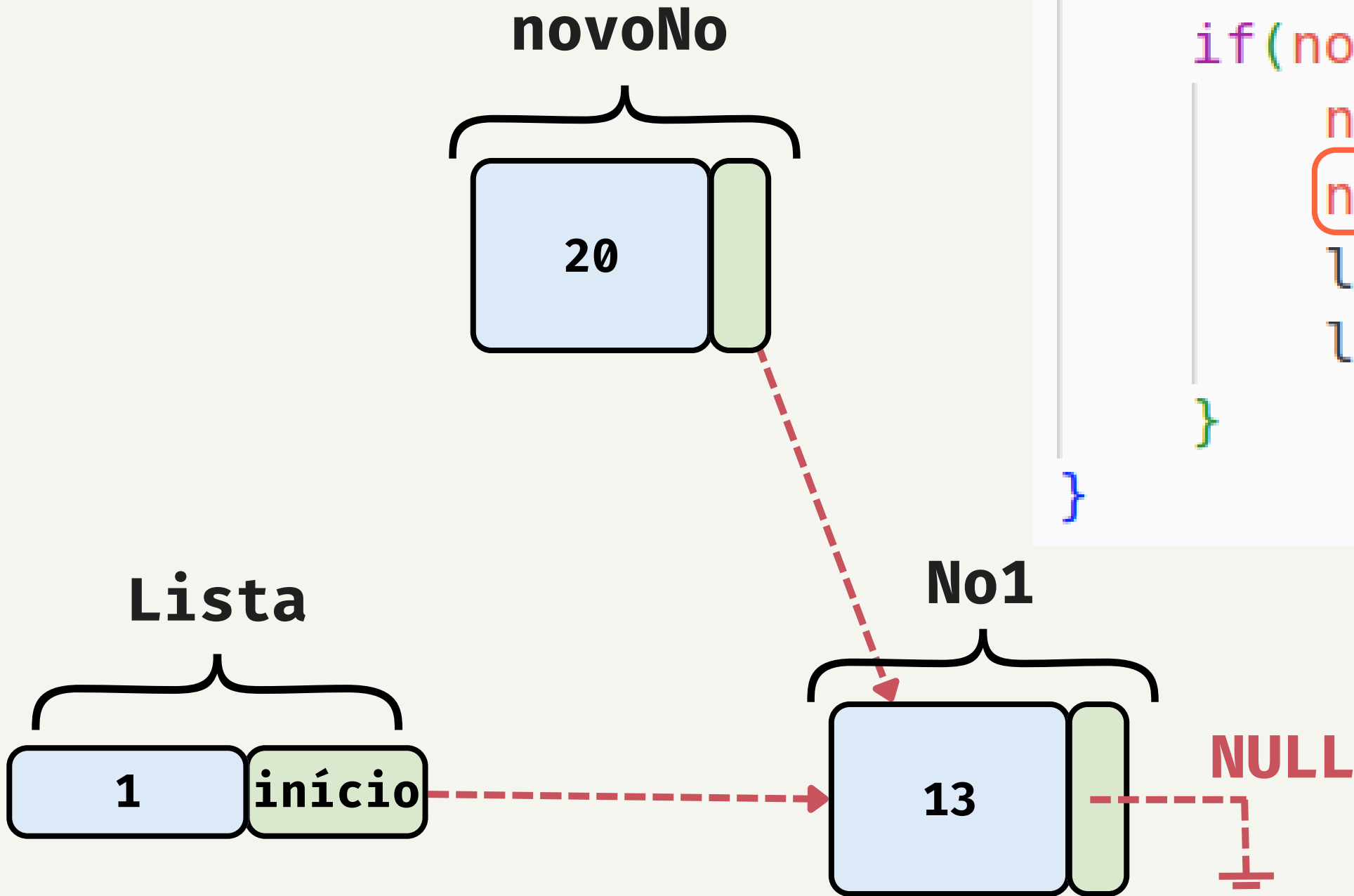
No1



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

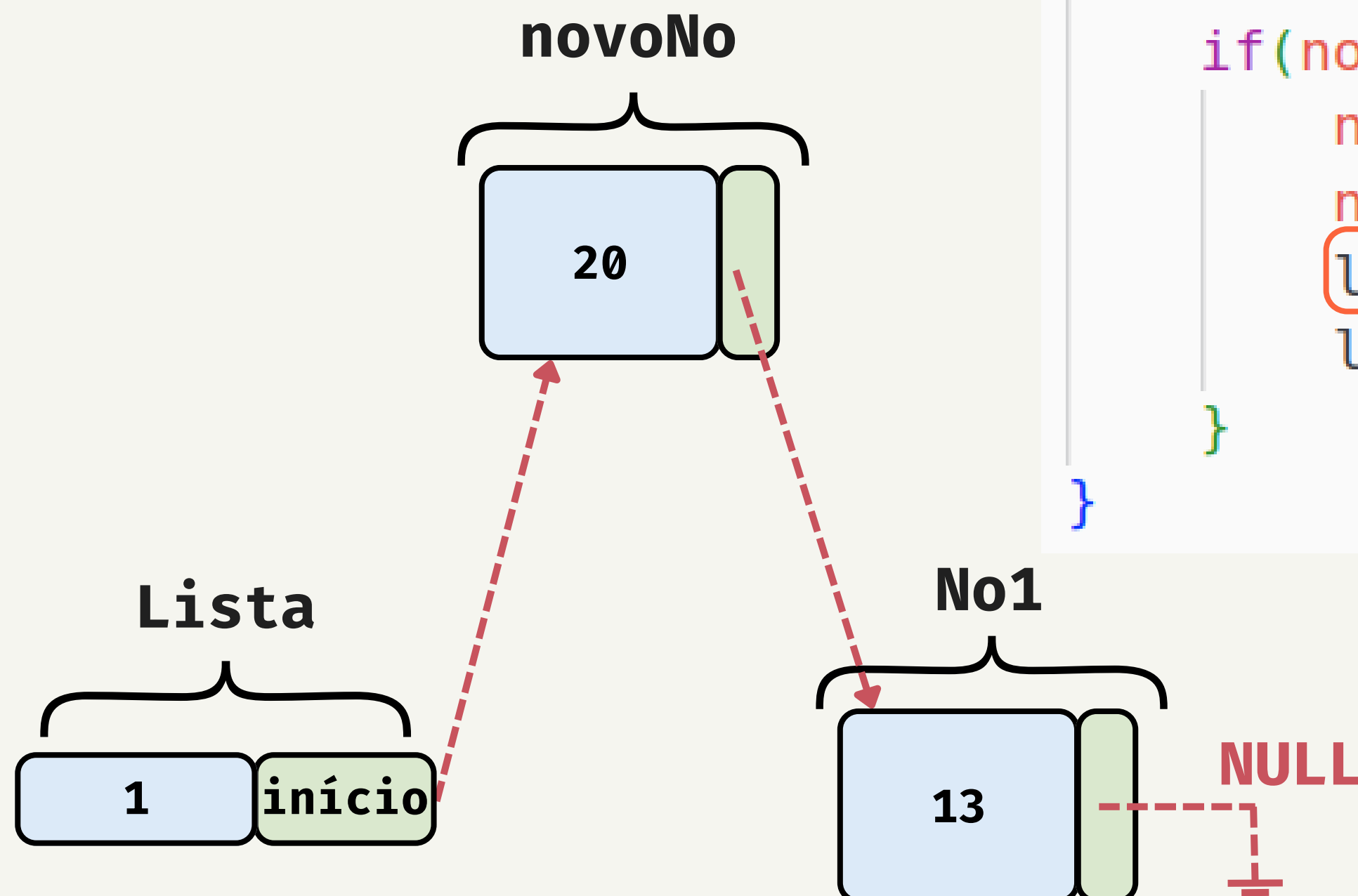
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

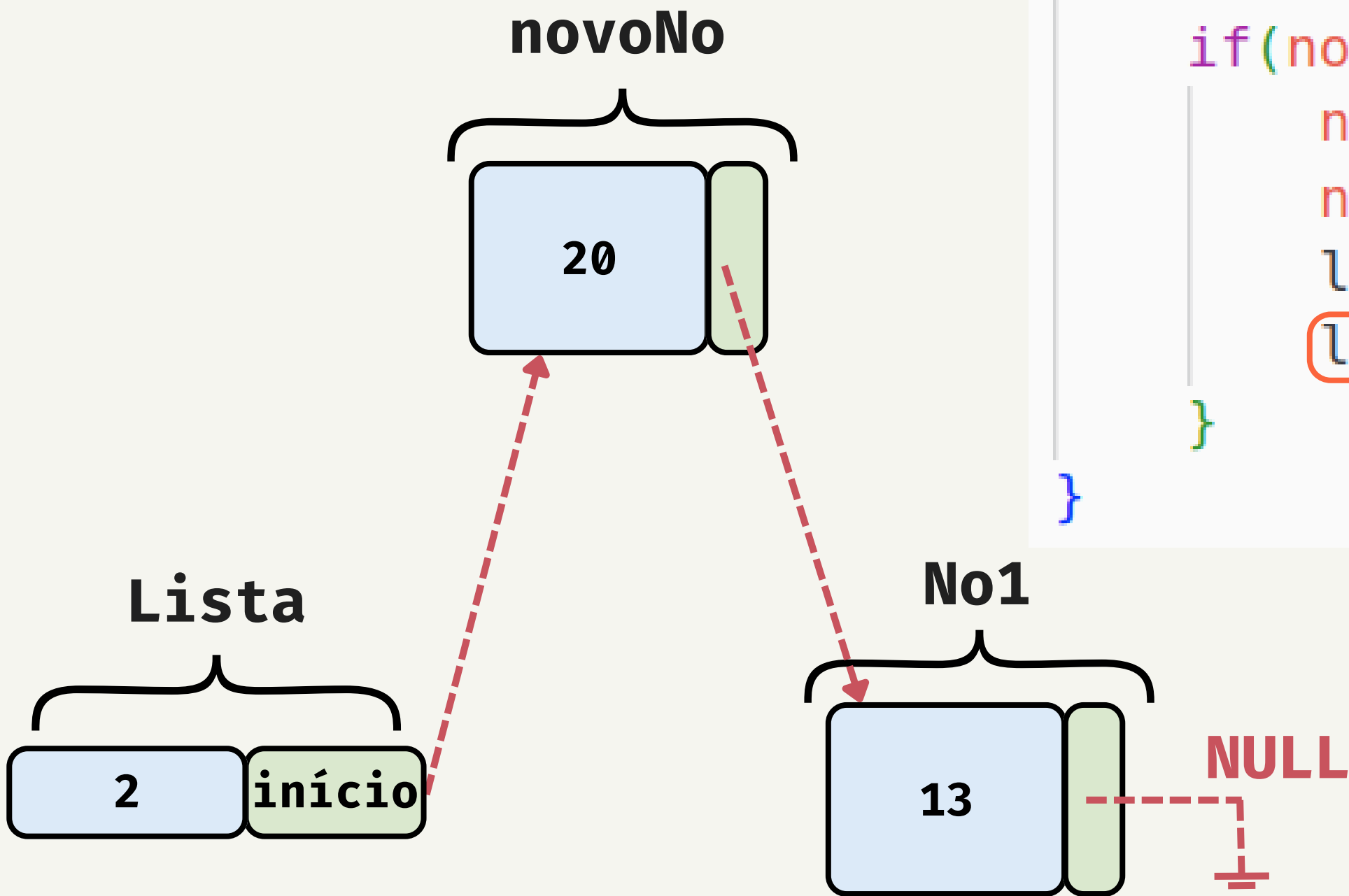
```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```



LISTA ENCADEADA SIMPLES

Função **inserirInicio()**

```
void inserirInicio(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = lista->inicio;
        lista->inicio = novoNo;
        lista->tamanho++;
    }
}
```

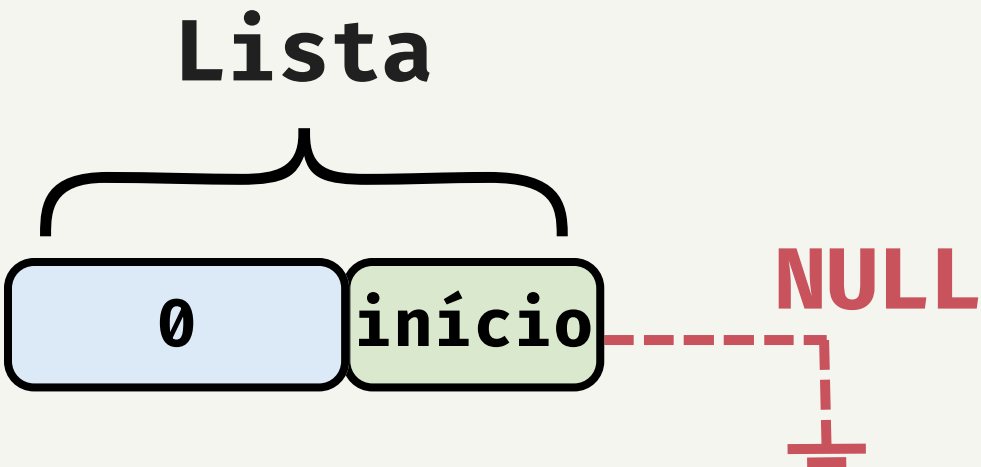


LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```



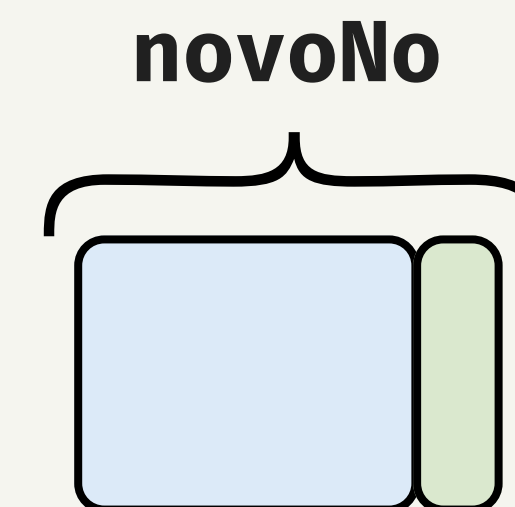
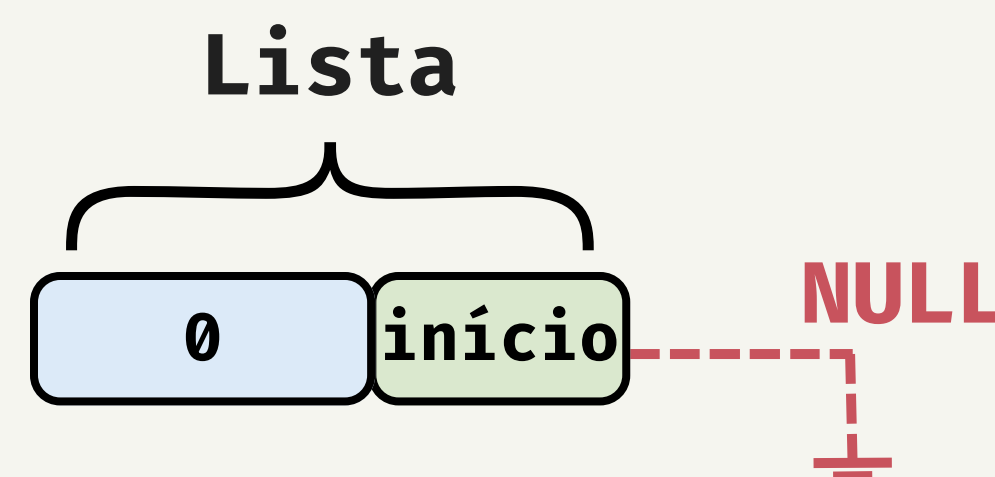
LISTA ENCADEADA SIMPLES

Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```



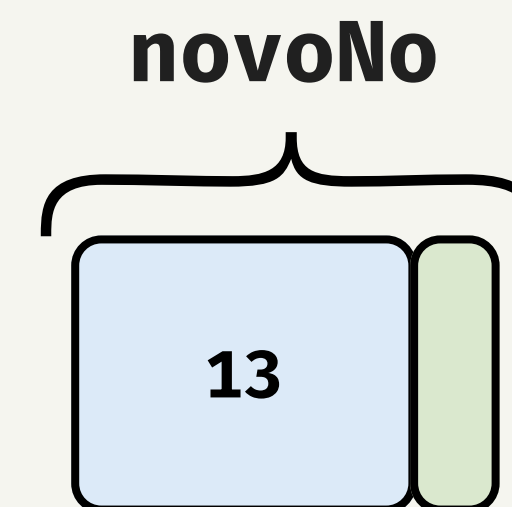
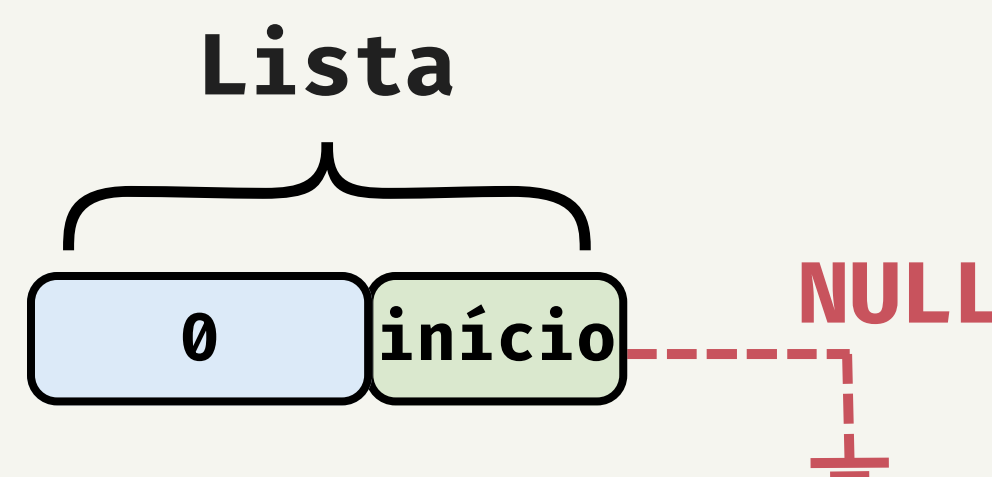
LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```



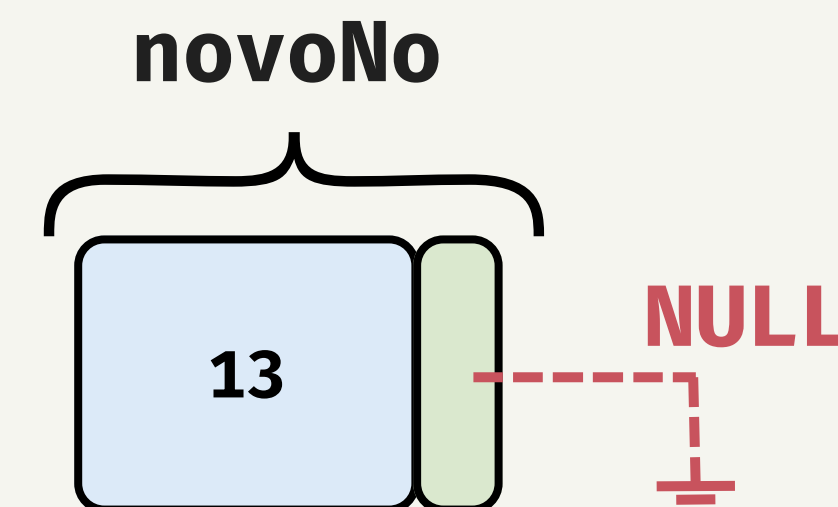
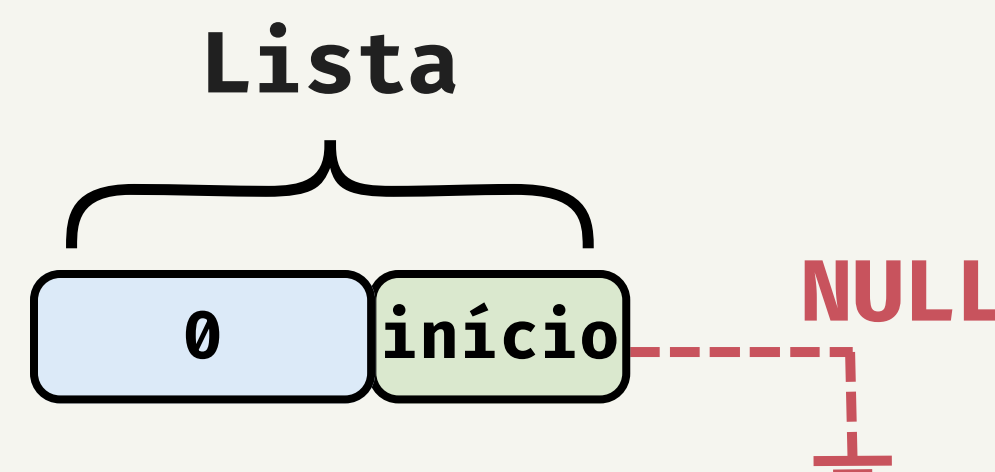
LISTA ENCADEADA SIMPLES

Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```



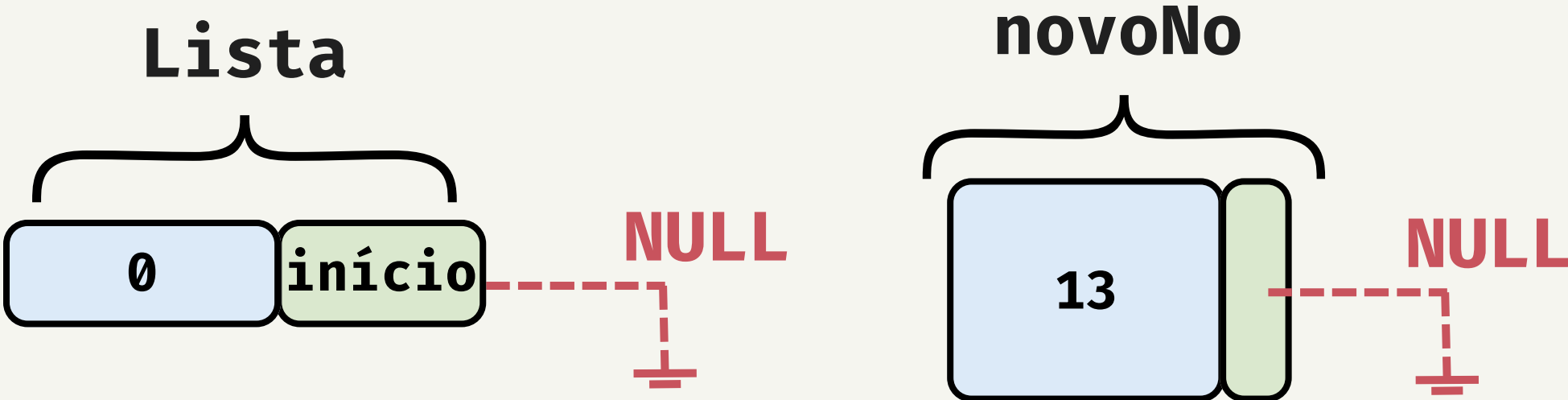
LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 1: Lista vazia



LISTA ENCADEADA SIMPLES

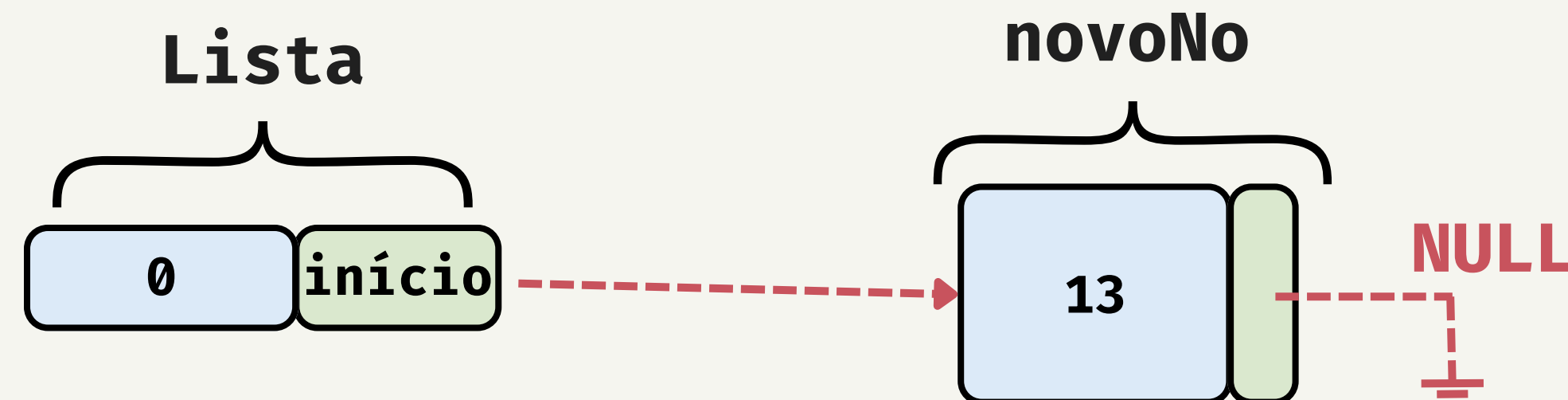
Função `inserirFim()`

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 1: Lista vazia



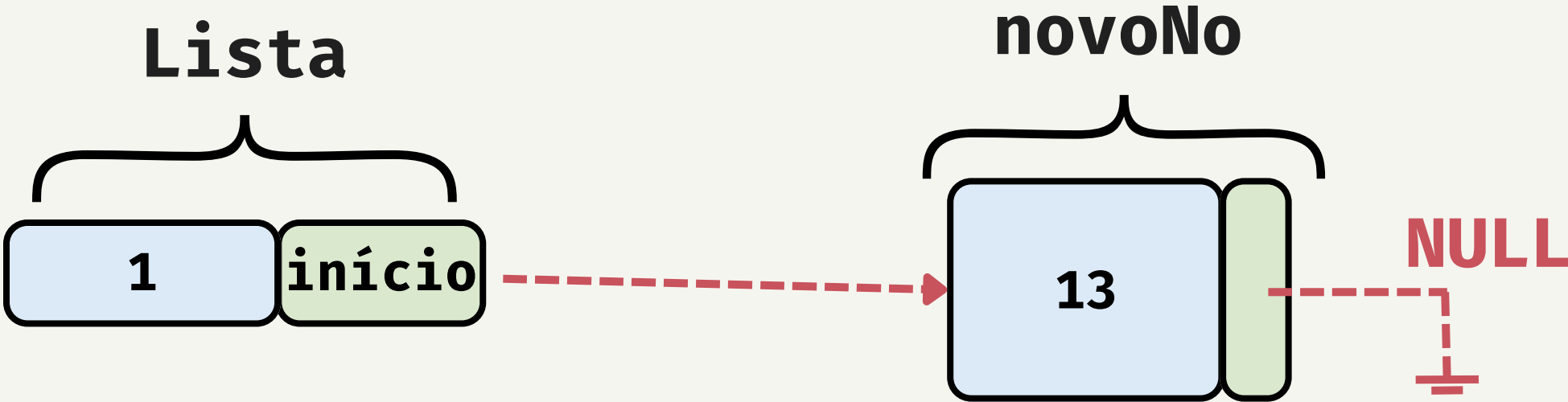
LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 1: Lista vazia



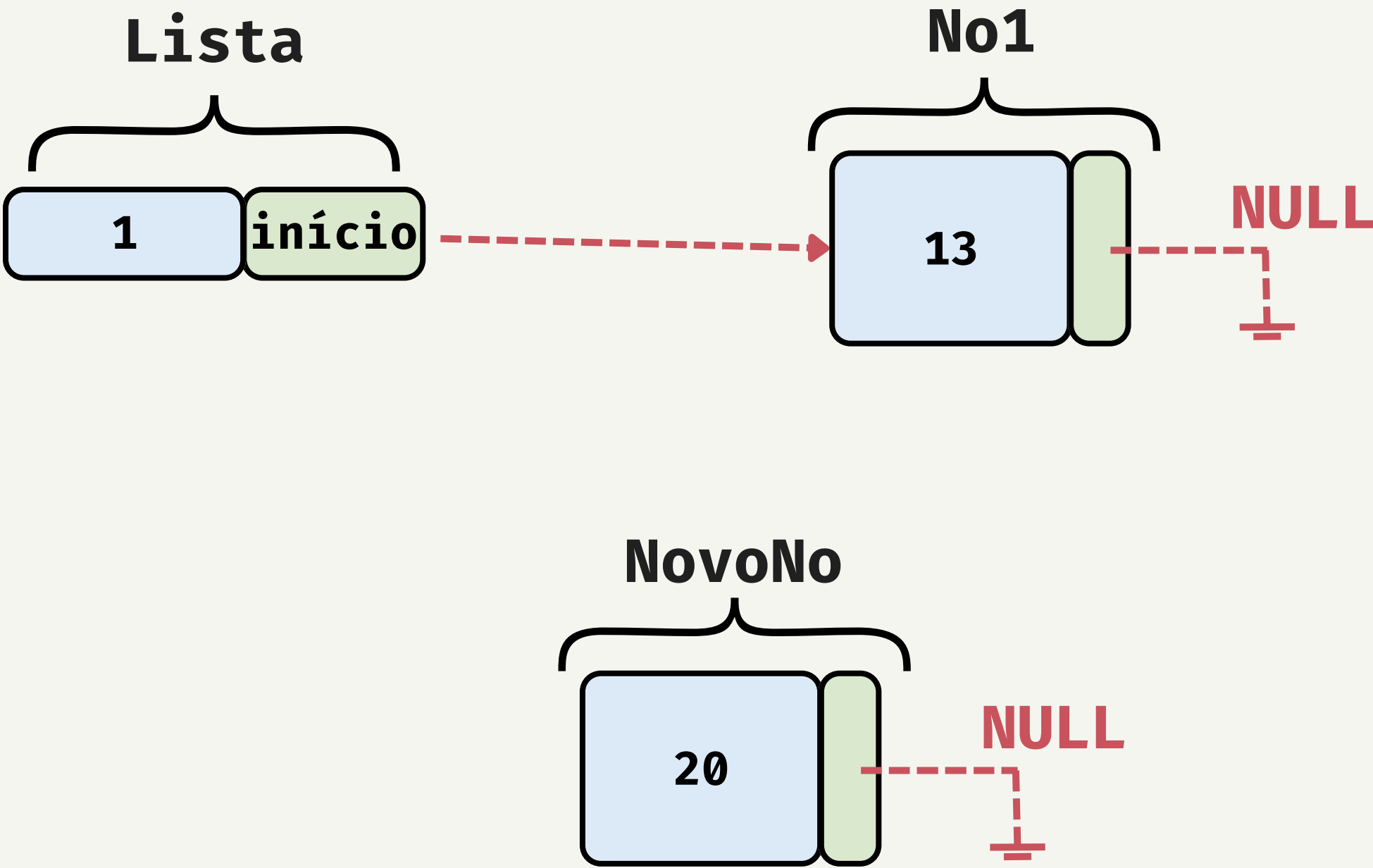
LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 2: Lista com 1 elemento



LISTA ENCADEADA SIMPLES

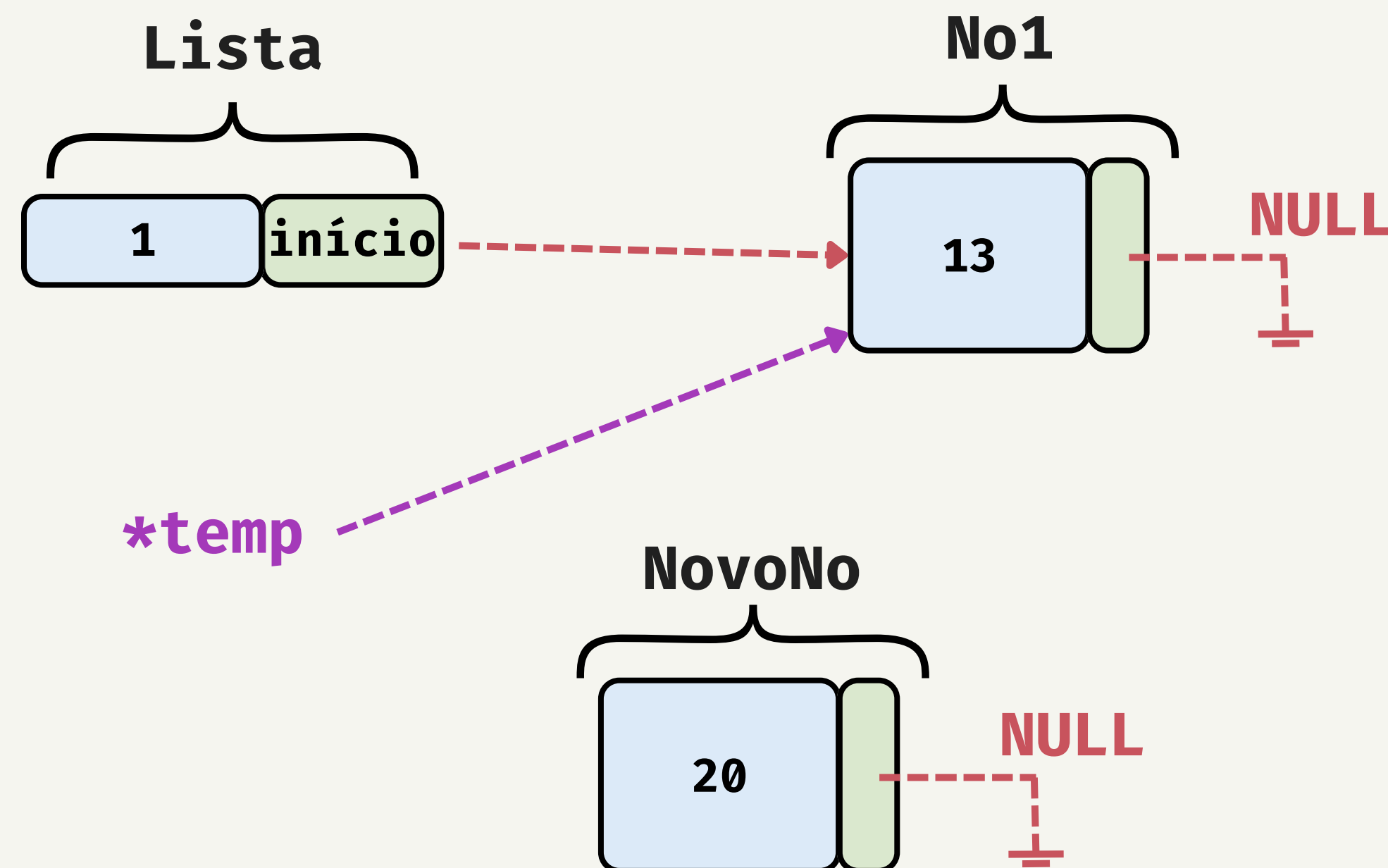
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 2: Lista com 1 elemento



LISTA ENCADEADA SIMPLES

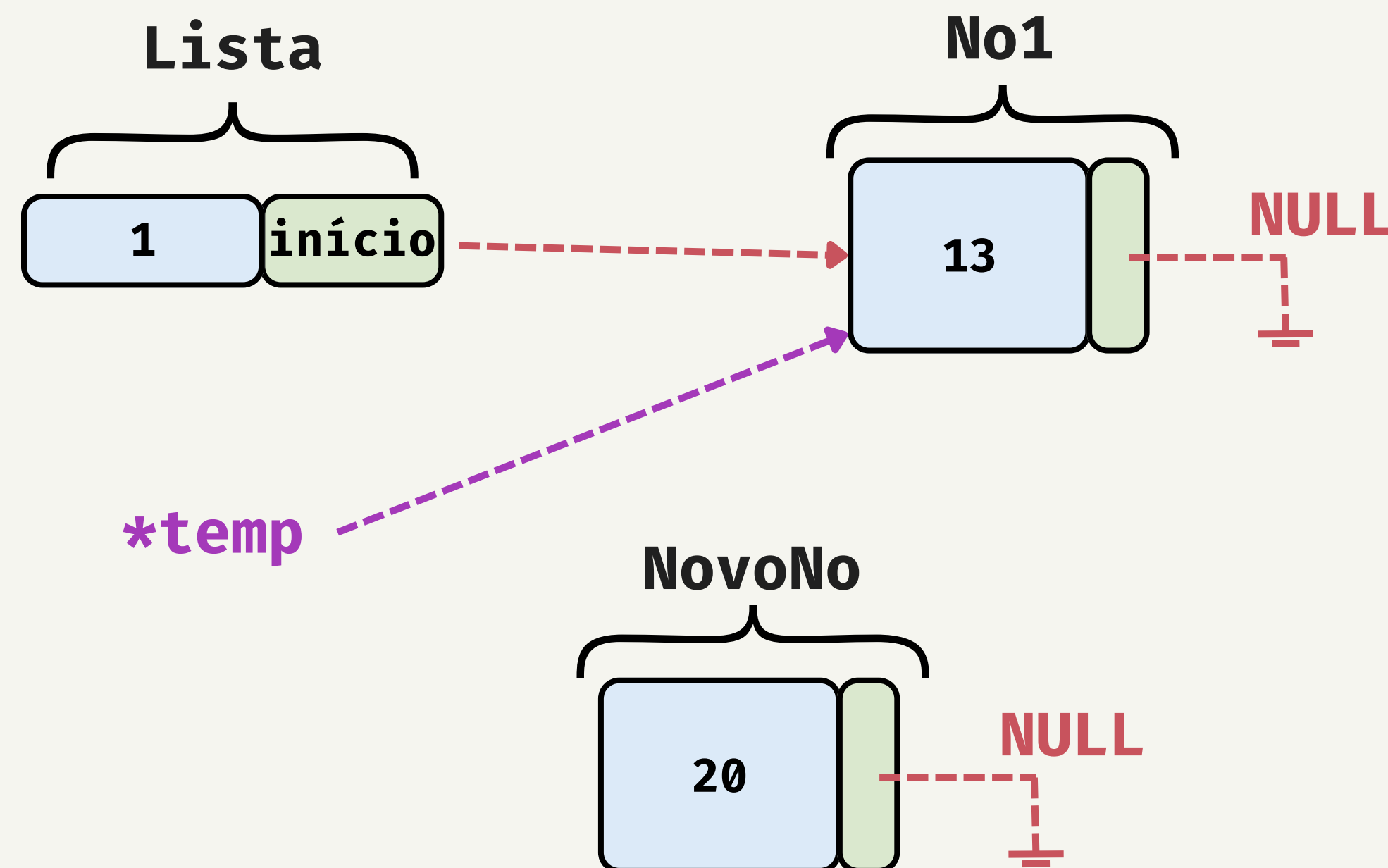
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 2: Lista com 1 elemento



LISTA ENCADEADA SIMPLES

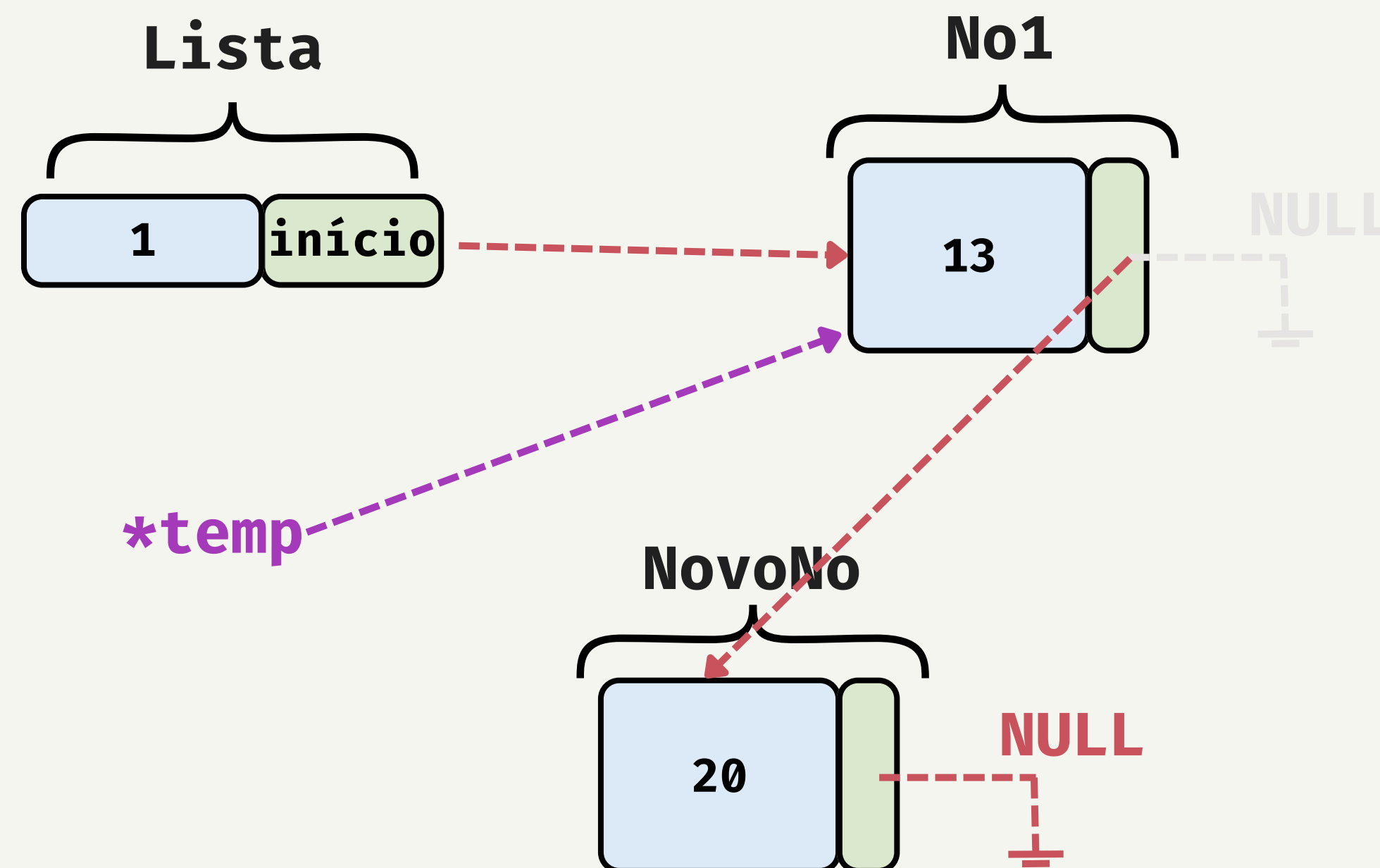
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 2: Lista com 1 elemento



LISTA ENCADEADA SIMPLES

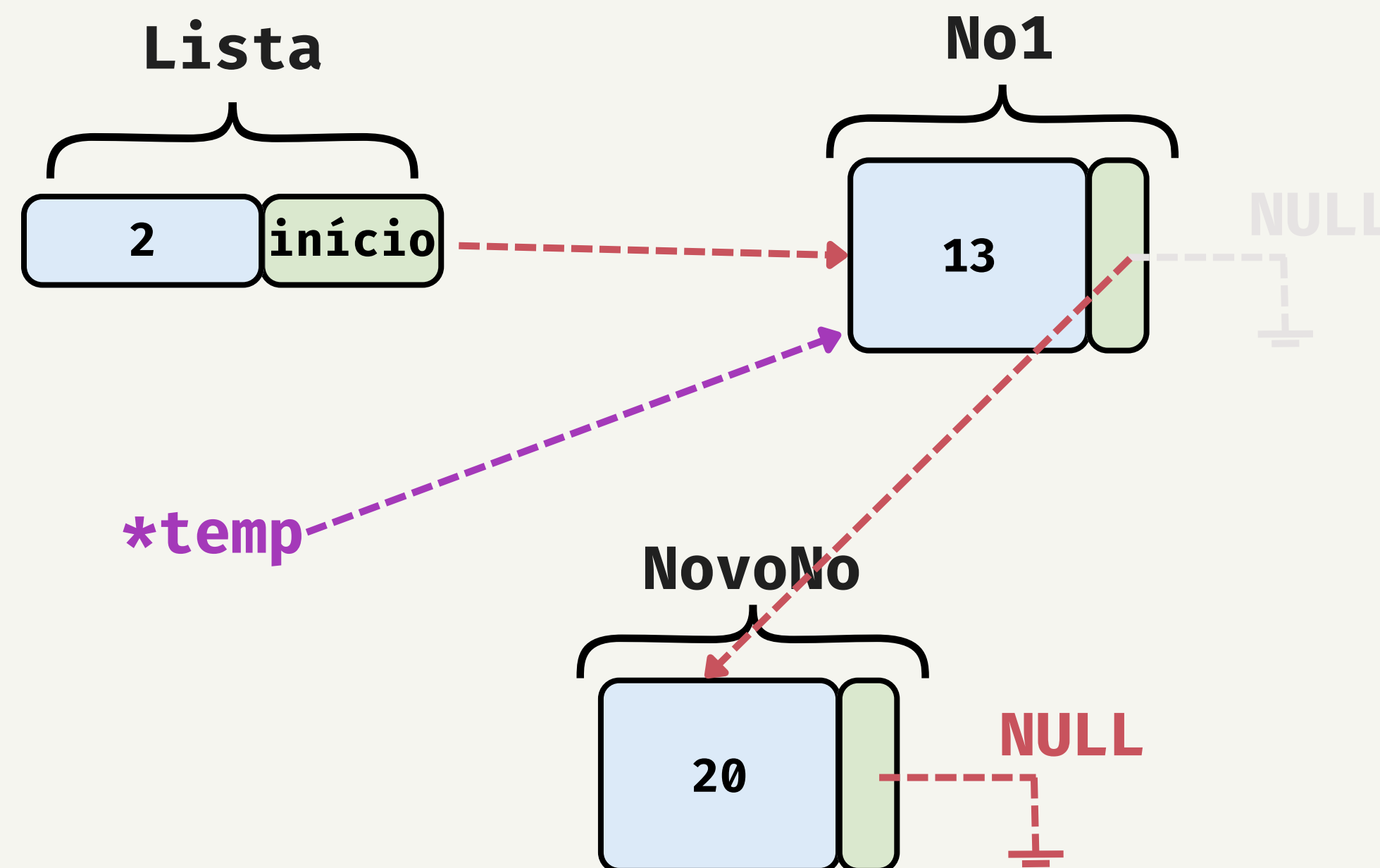
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 2: Lista com 1 elemento



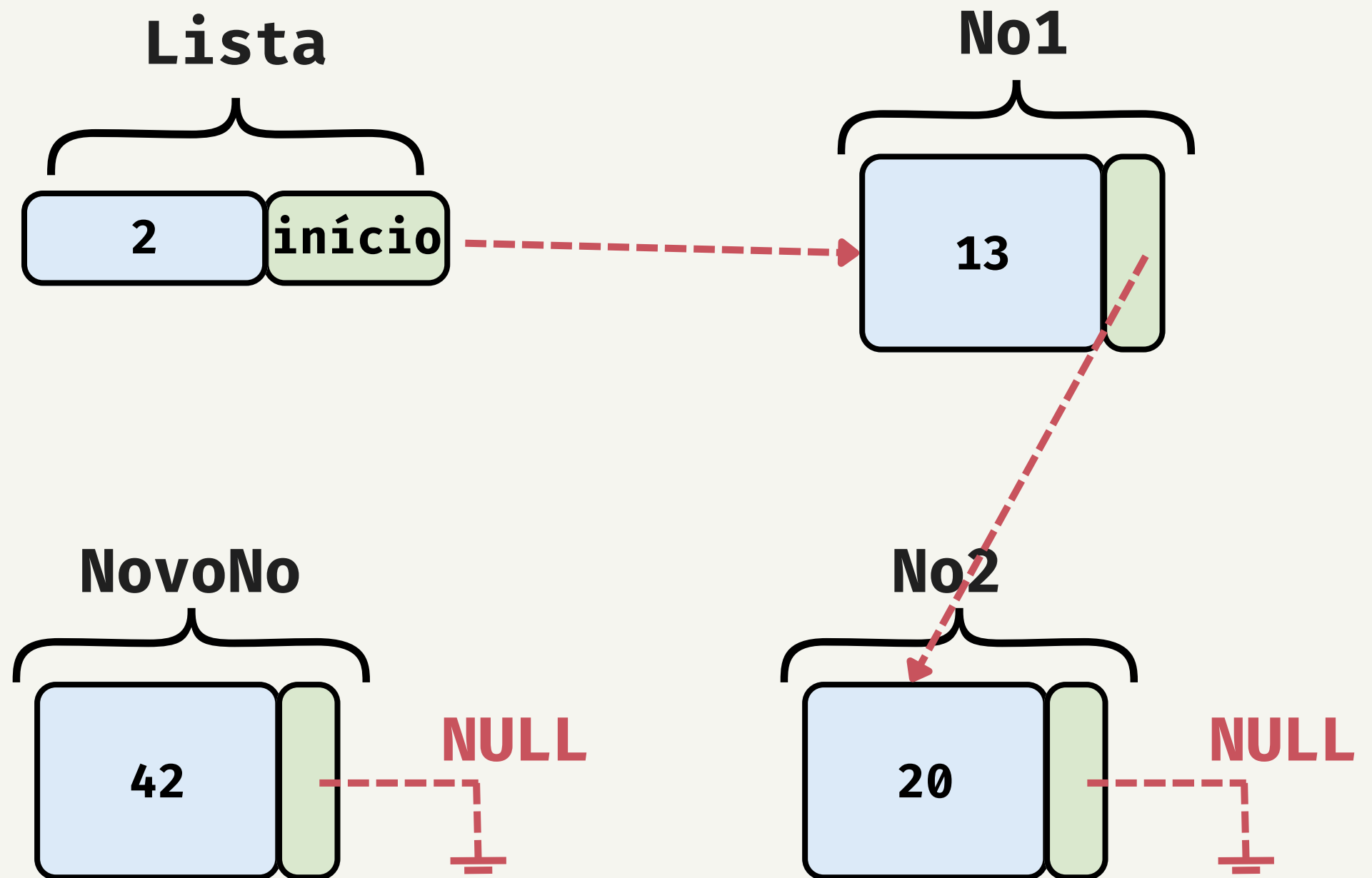
LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 3: Lista com 2 elementos



LISTA ENCADEADA SIMPLES

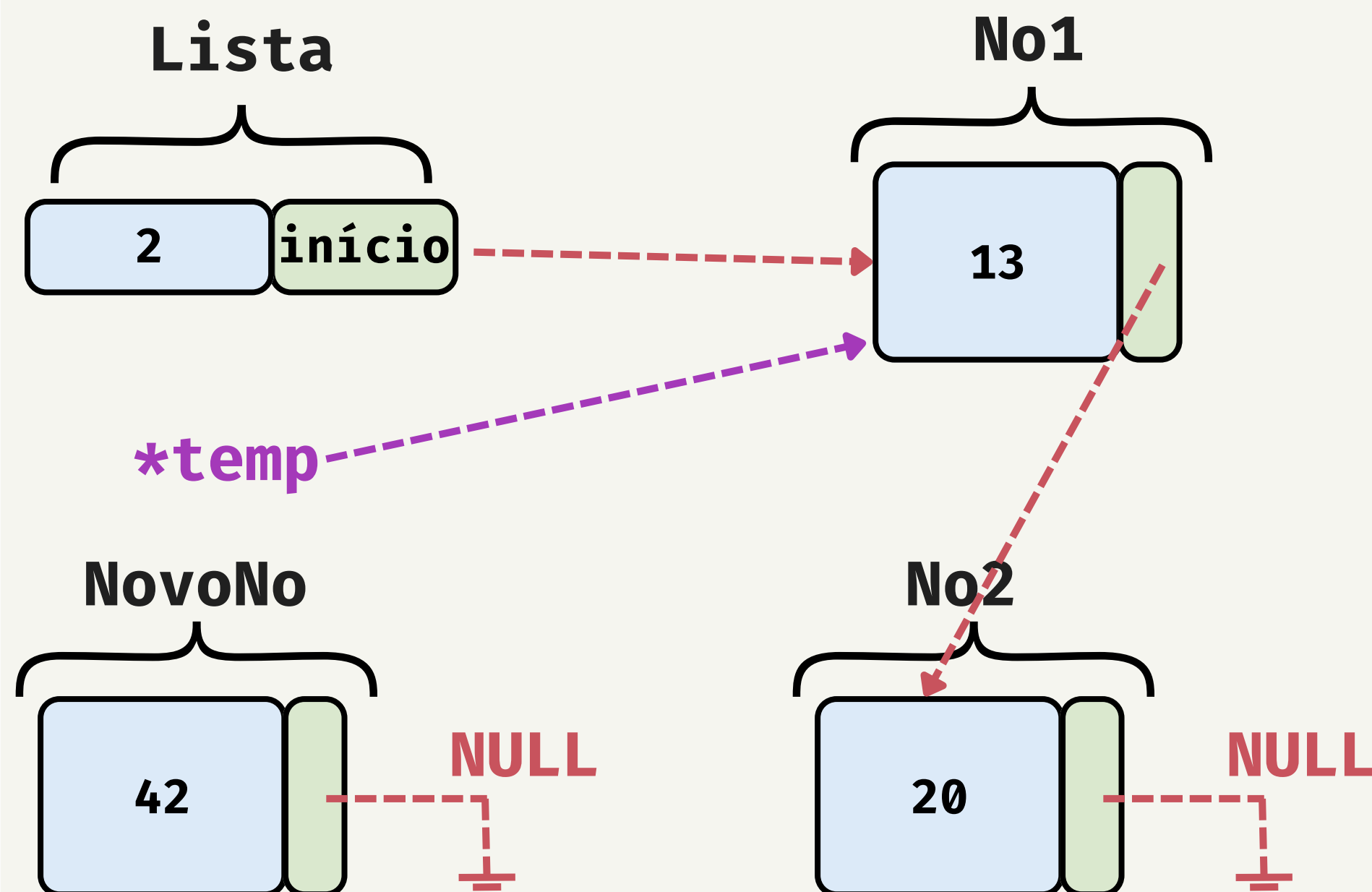
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 3: Lista com 2 elementos



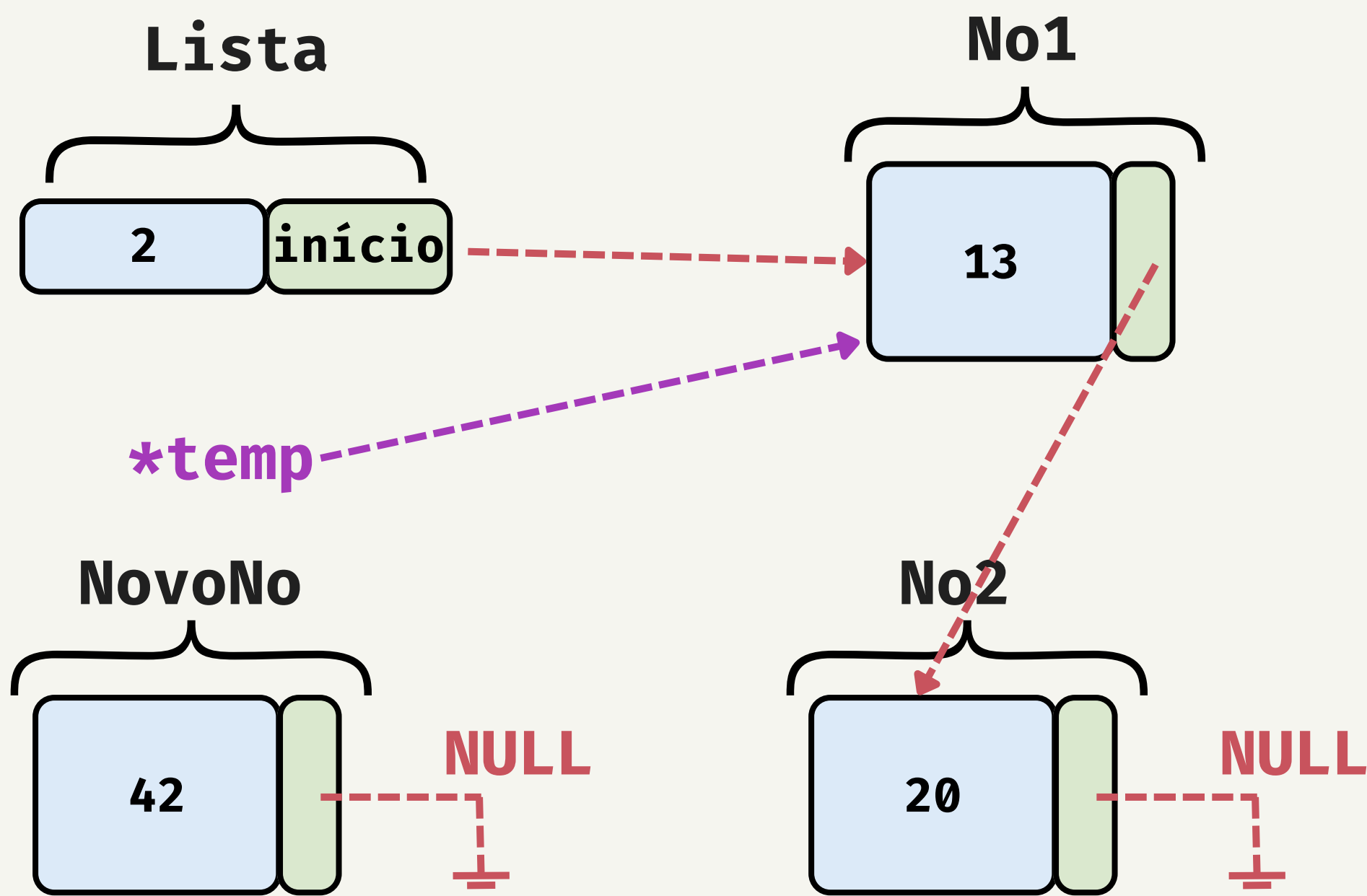
LISTA ENCADEADA SIMPLES

Função **inserirFim()**

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 3: Lista com 2 elementos



LISTA ENCADEADA SIMPLES

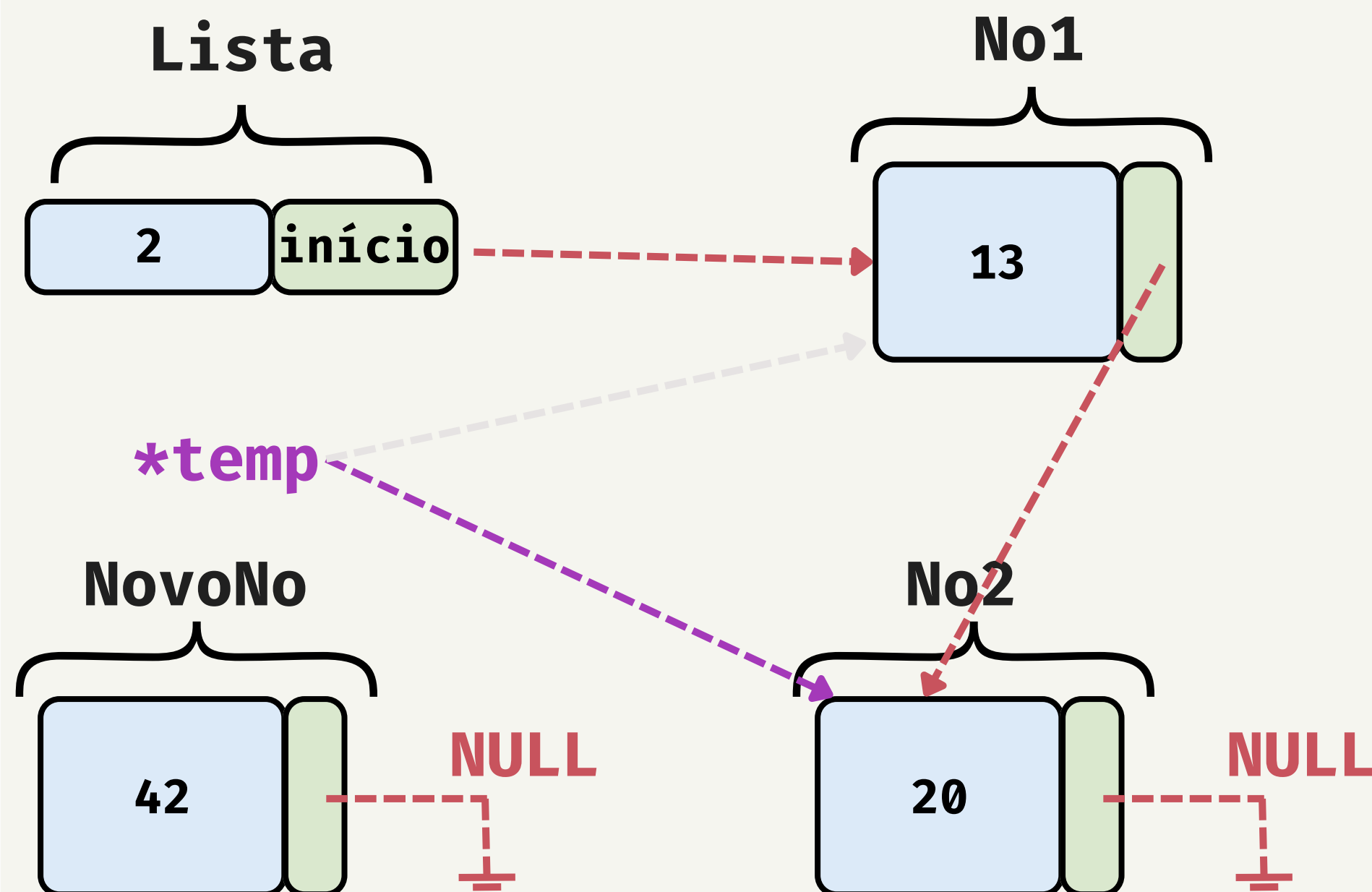
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 3: Lista com 2 elementos



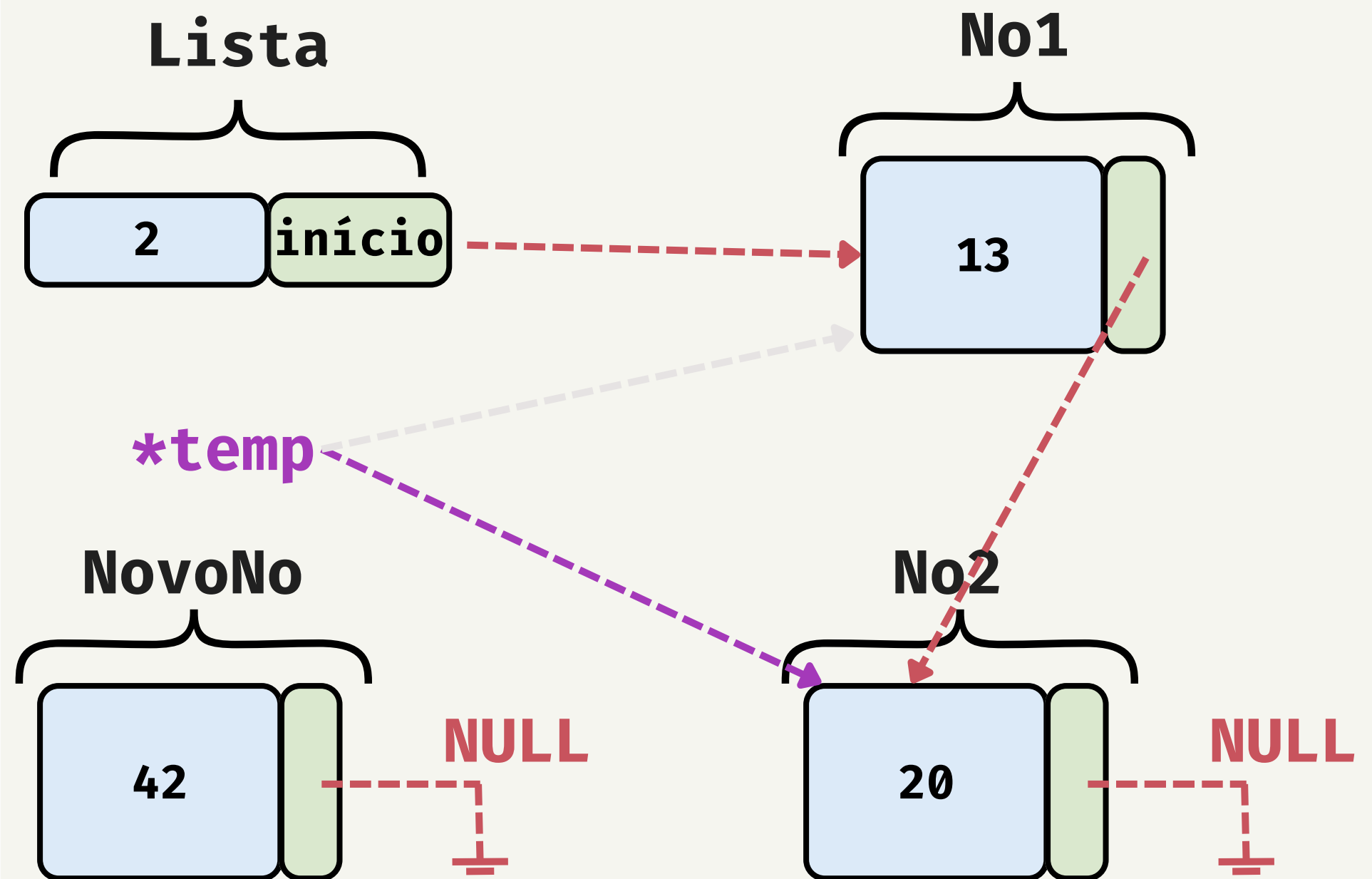
LISTA ENCADEADA SIMPLES

Função `inserirFim()`

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 3: Lista com 2 elementos



LISTA ENCADEADA SIMPLES

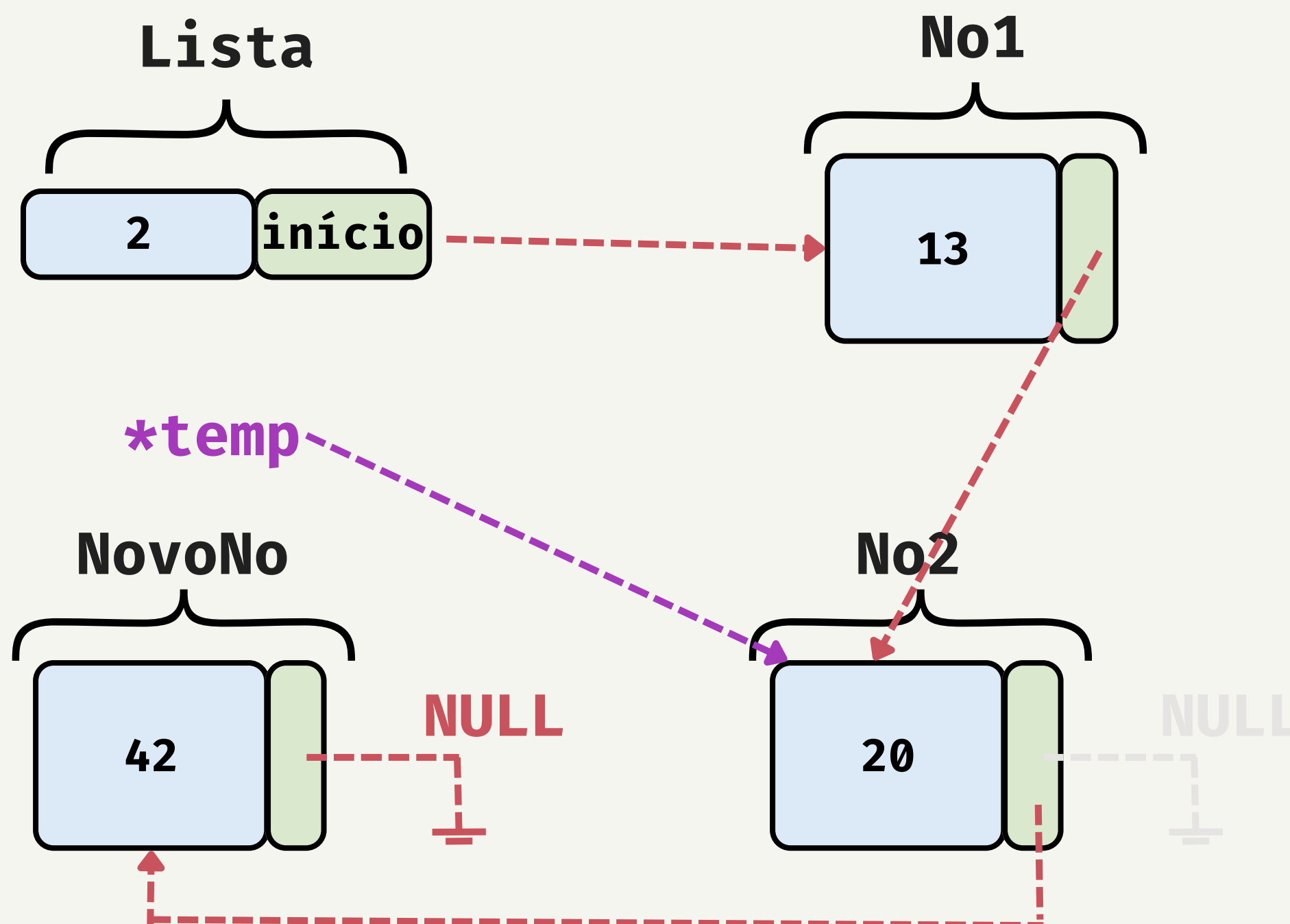
Função **inserirFim()**

```

void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
    
```

Caso 3: Lista com 2 elementos



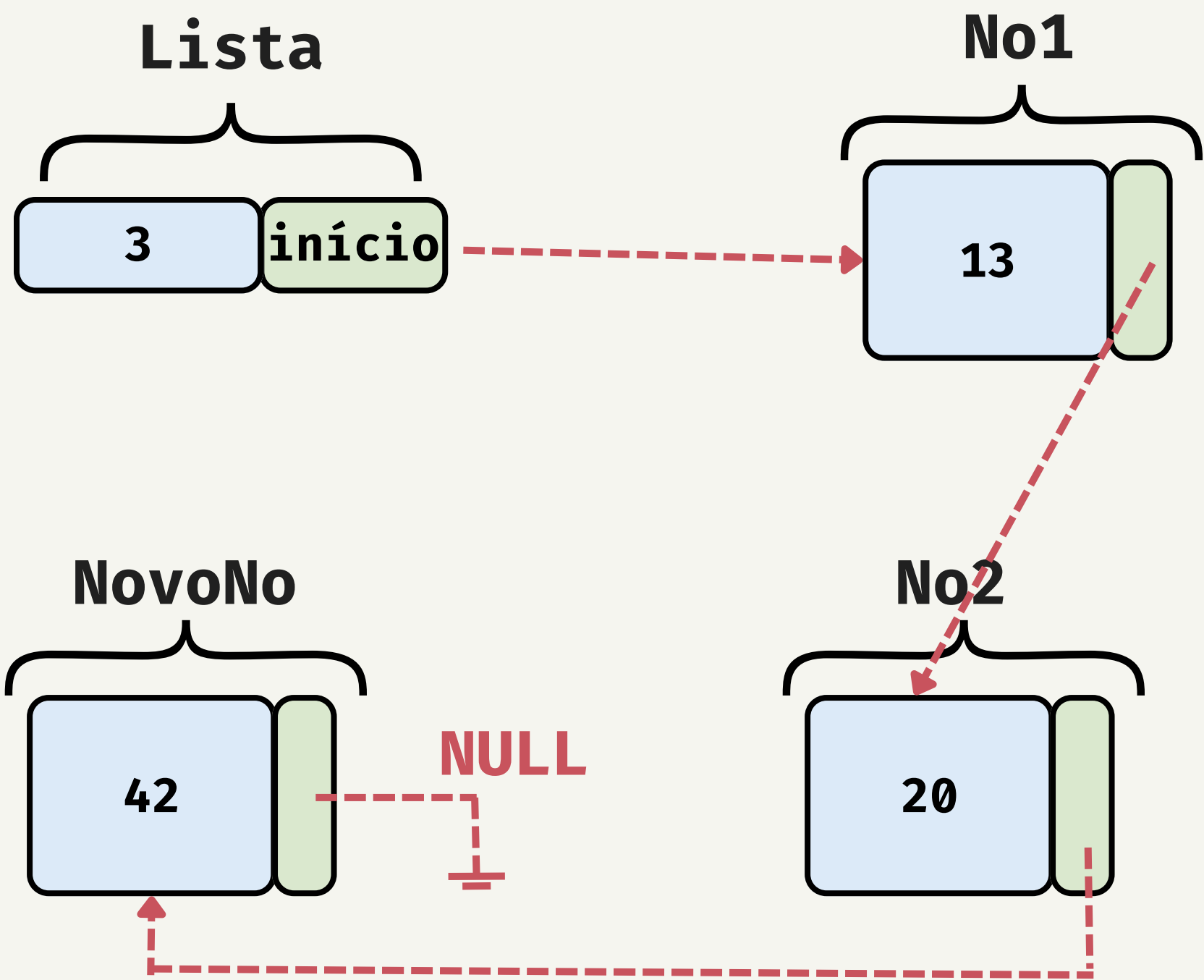
LISTA ENCADEADA SIMPLES

Função **inserirFim()**

```
void inserirFim(Lista* lista, int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    if(novoNo){
        novoNo->valorNo = valor;
        novoNo->proxNo = NULL;

        if(lista->inicio == NULL) {
            lista->inicio = novoNo;
        } else {
            No* temp = lista->inicio;
            while(temp->proxNo != NULL) {
                temp = temp->proxNo;
            }
            temp->proxNo = novoNo;
        }
        lista->tamanho++;
    }
}
```

Caso 3: Lista com 2 elementos

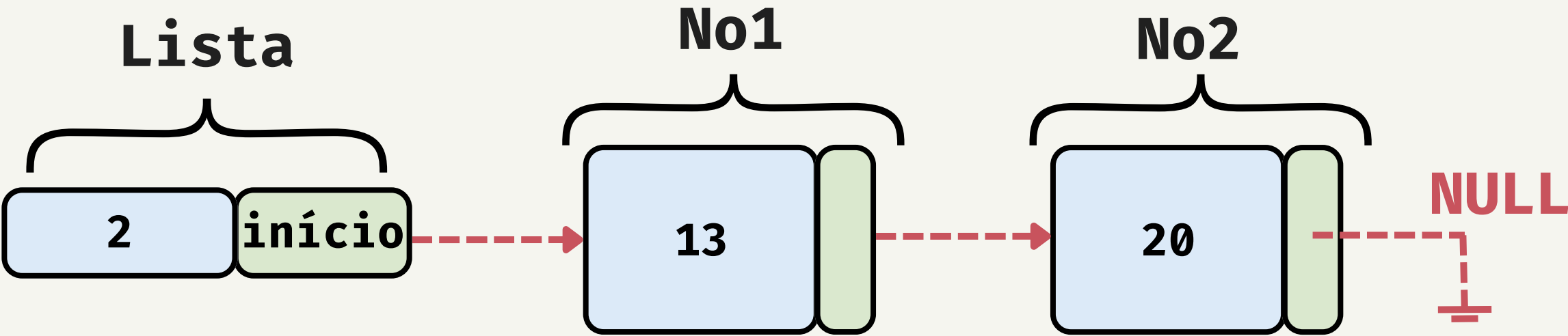


LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

Número para
pesquisar: 20

```
No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
```



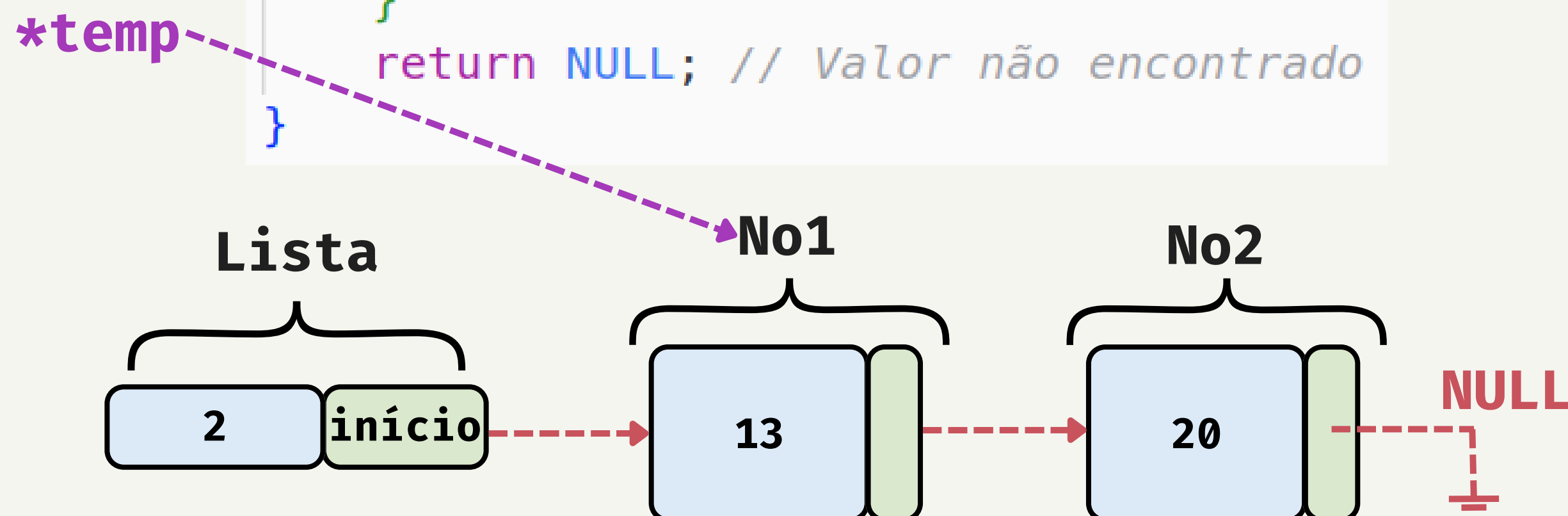
LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

```

No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
    
```

Número para
pesquisar: 20



LISTA ENCADEADA SIMPLES

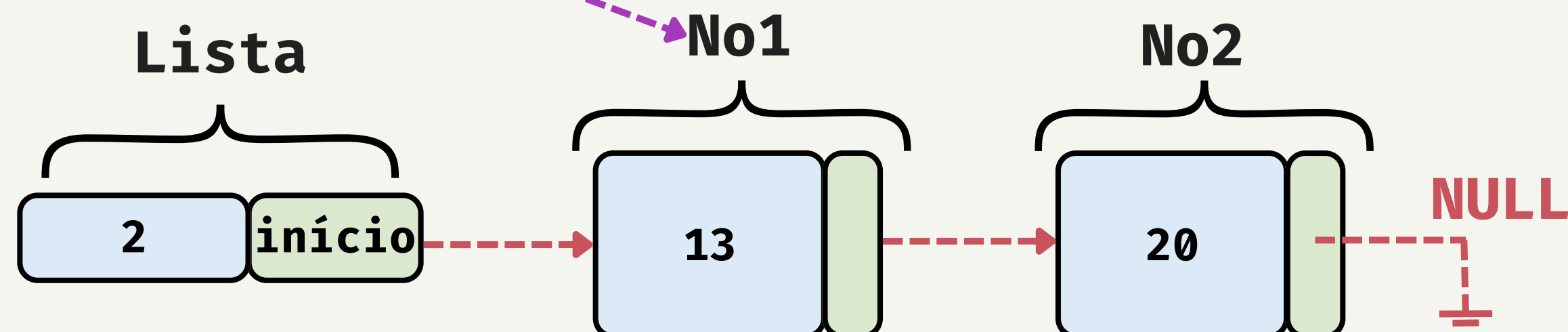
Função **pesquisar()** ou **buscar()**

```

No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
    
```

Número para
pesquisar: 20

**temp*



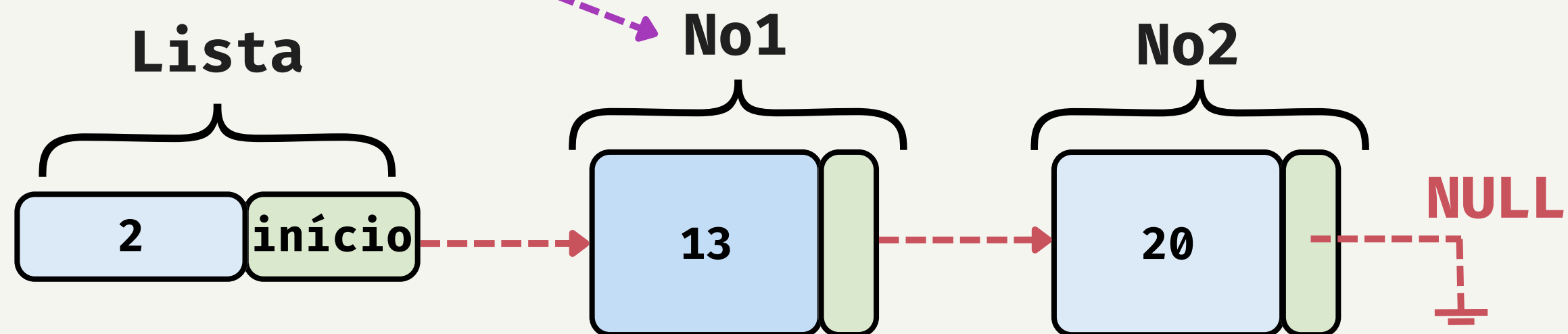
LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

Número para
pesquisar: 20

```
No* buscar(Lista* lista, int valor) {  
    No* temp = lista->inicio;  
    while(temp != NULL) {  
        if(temp->valorNo == valor) {  
            return temp;  
        }  
        temp = temp->proxNo;  
    }  
    return NULL; // Valor não encontrado  
}
```

**temp*

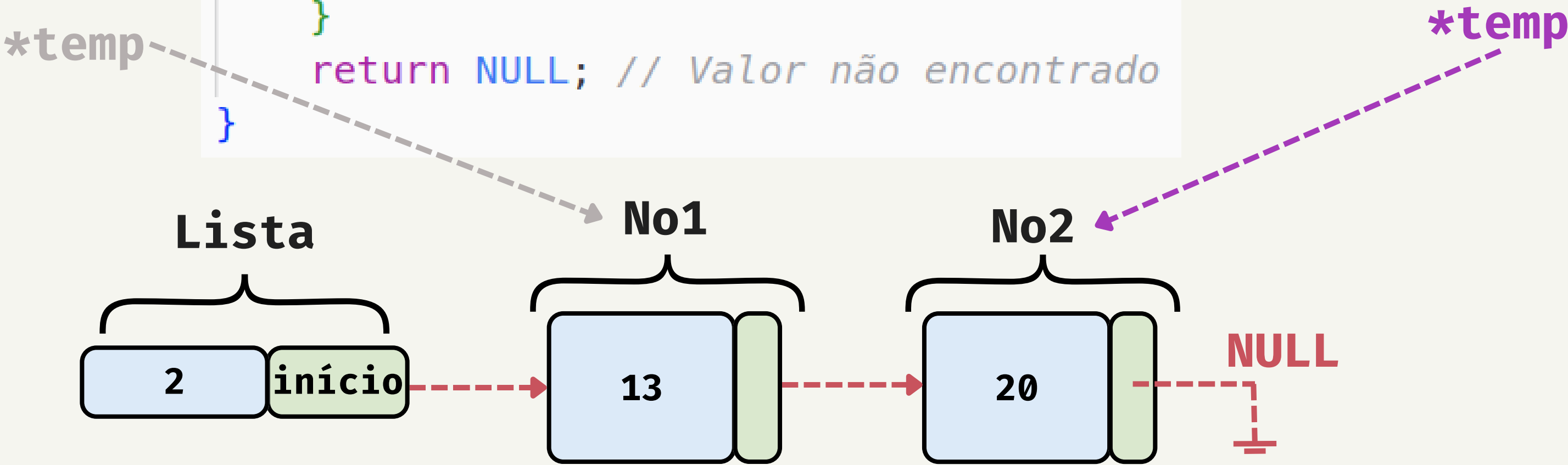


LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

Número para
pesquisar: 20

```
No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
```



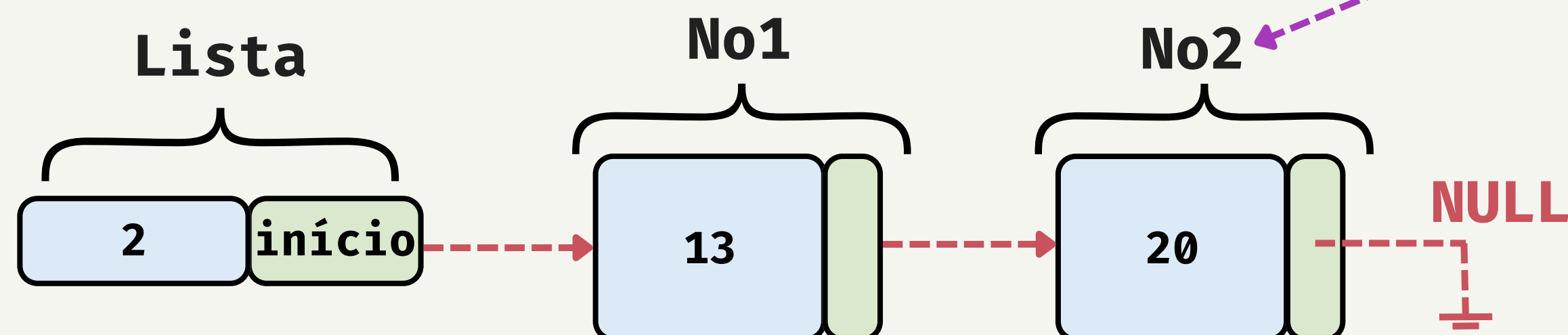
LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

```

No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
    
```

Número para
pesquisar: 20



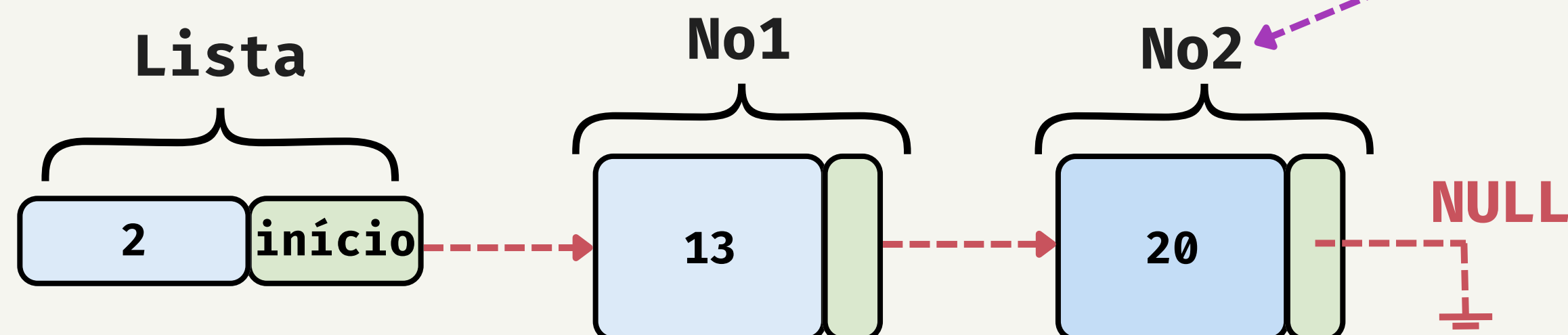
LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

```

No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
    
```

Número para
pesquisar: 20

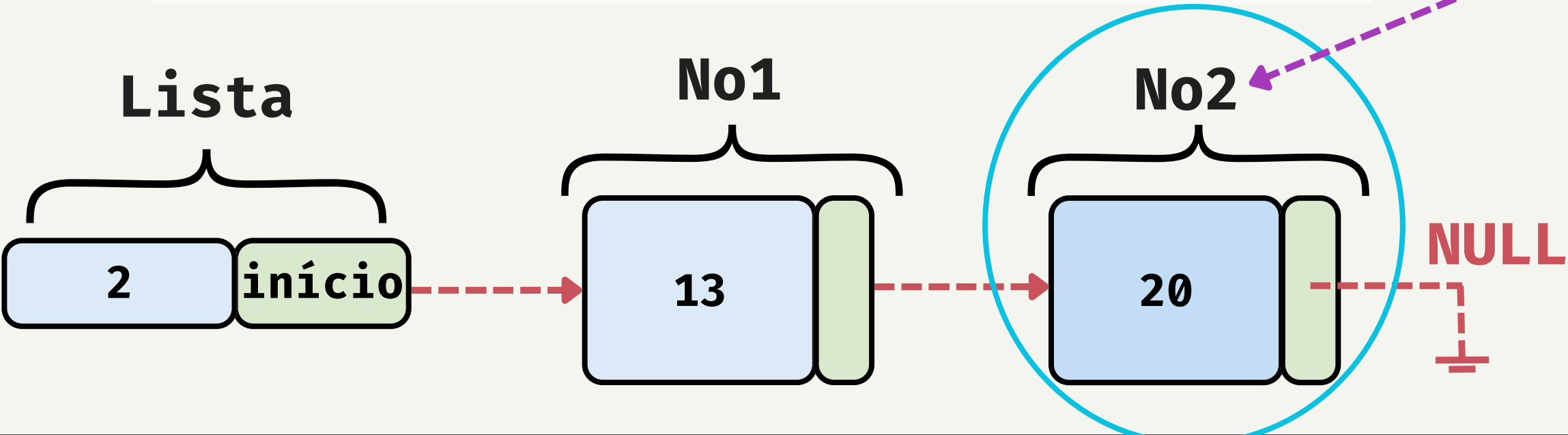


LISTA ENCADEADA SIMPLES

Função **pesquisar()** ou **buscar()**

Número para
pesquisar: 20

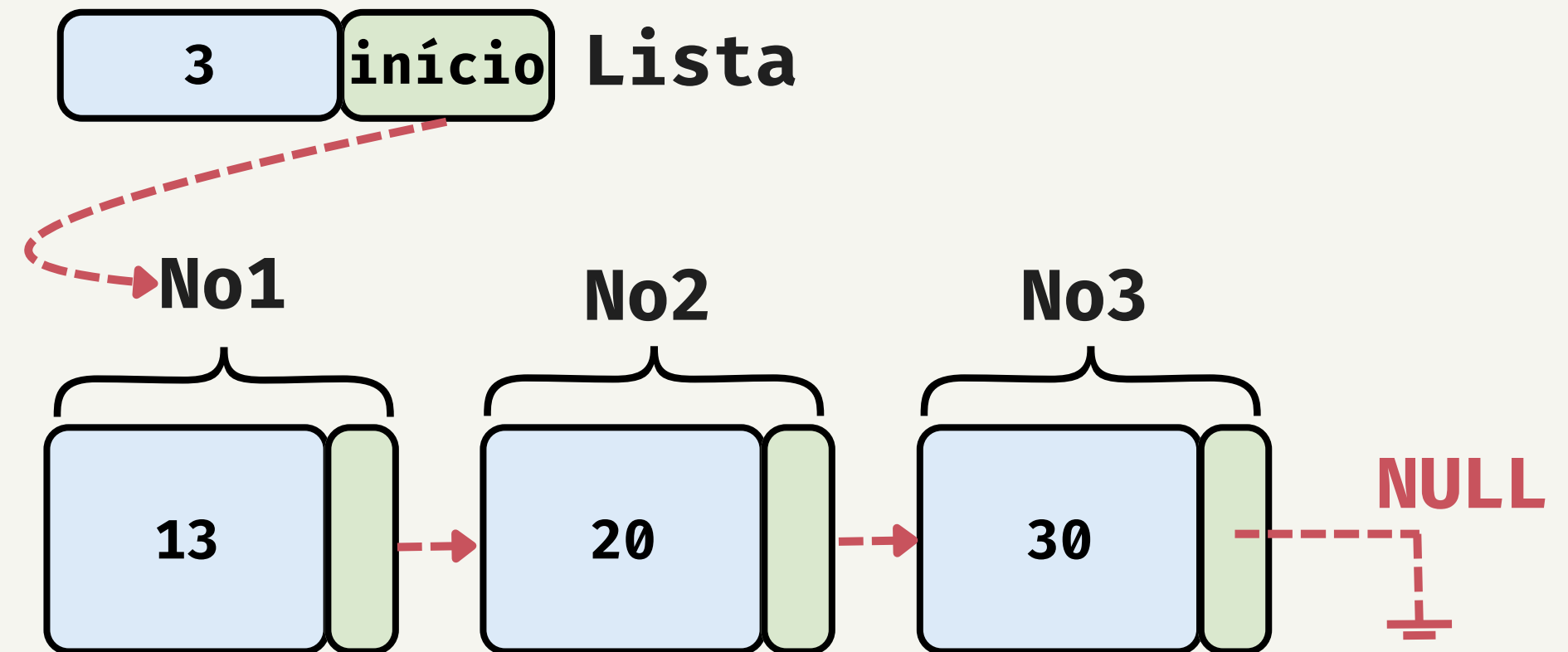
```
No* buscar(Lista* lista, int valor) {
    No* temp = lista->inicio;
    while(temp != NULL) {
        if(temp->valorNo == valor) {
            return temp;
        }
        temp = temp->proxNo;
    }
    return NULL; // Valor não encontrado
}
```



LISTA ENCADEADA SIMPLES

Função **Remove()**

```
No *remove(Lista* lista, int valor)
```



LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista

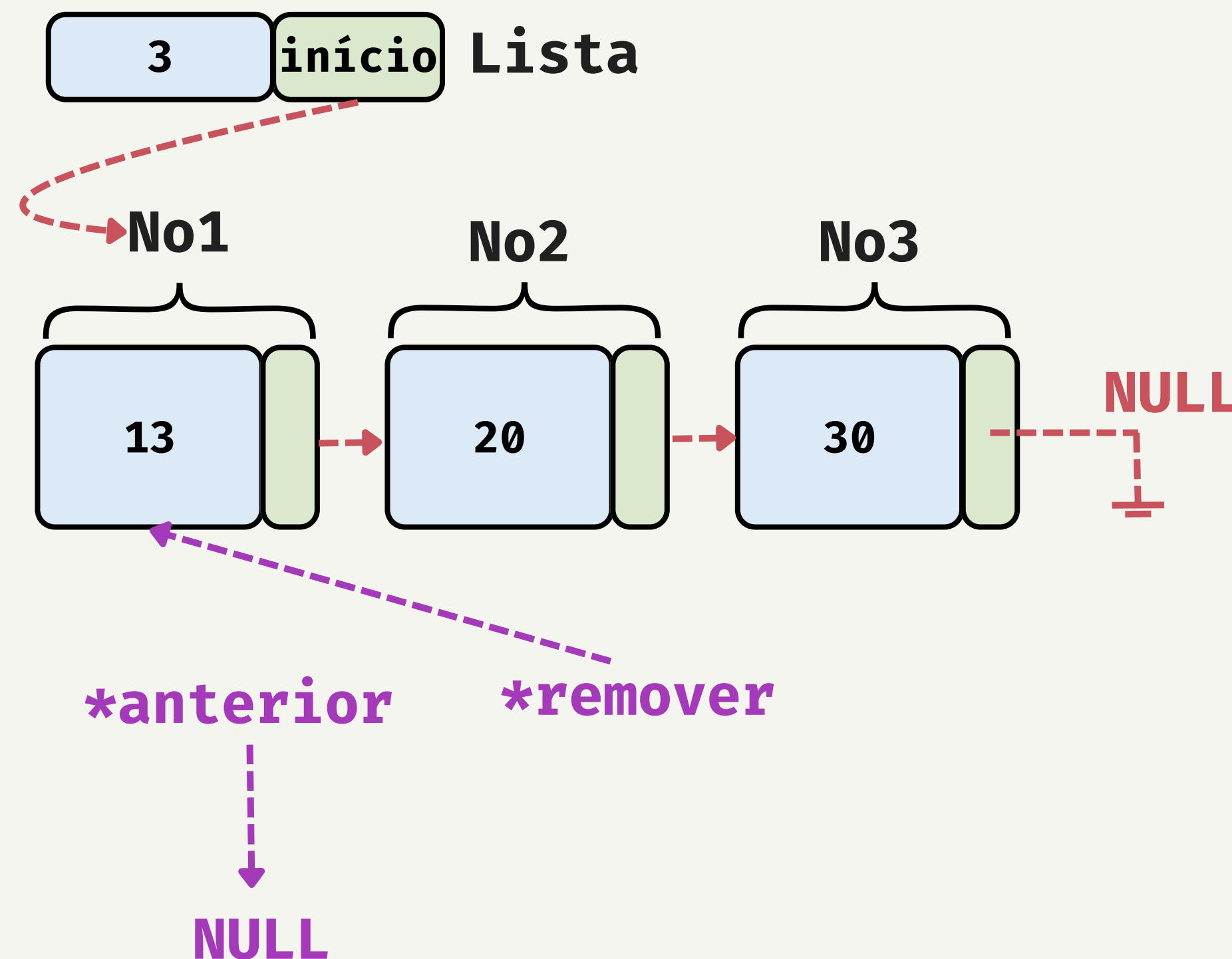
Remover valor = 13

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
    
```



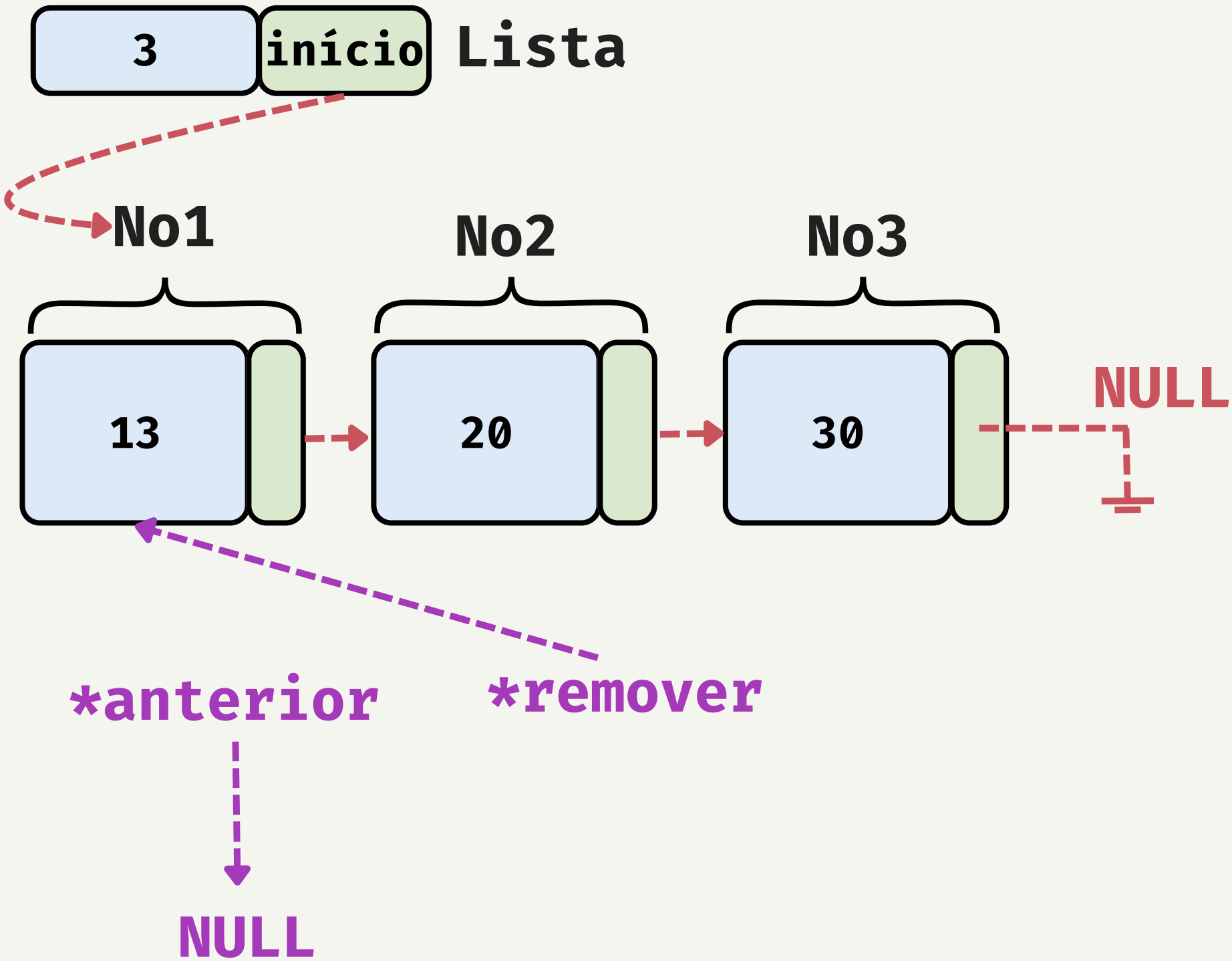
LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista
Remover valor = 13

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista

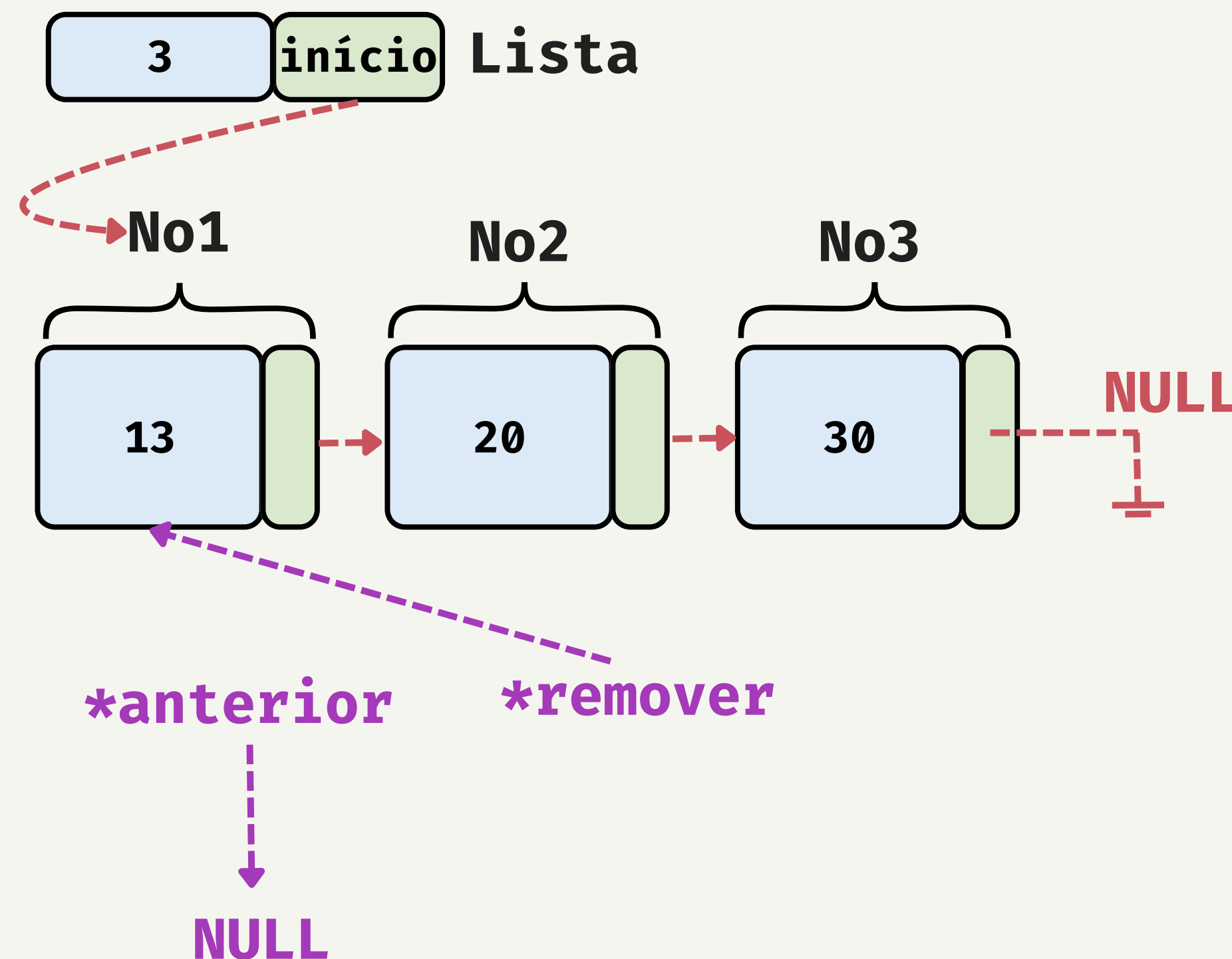
Remover valor = 13

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
    
```



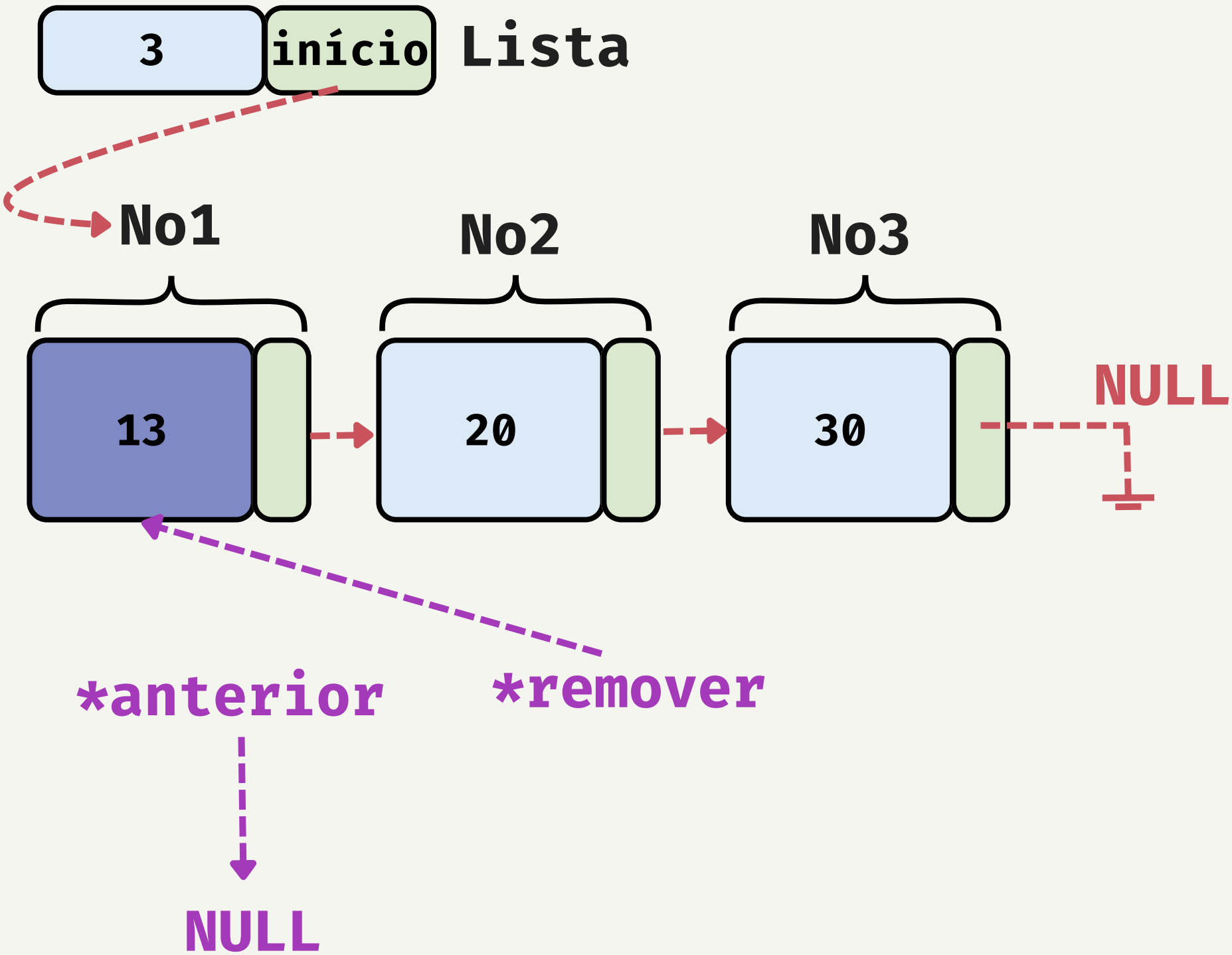
LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista
Remover valor = 13

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



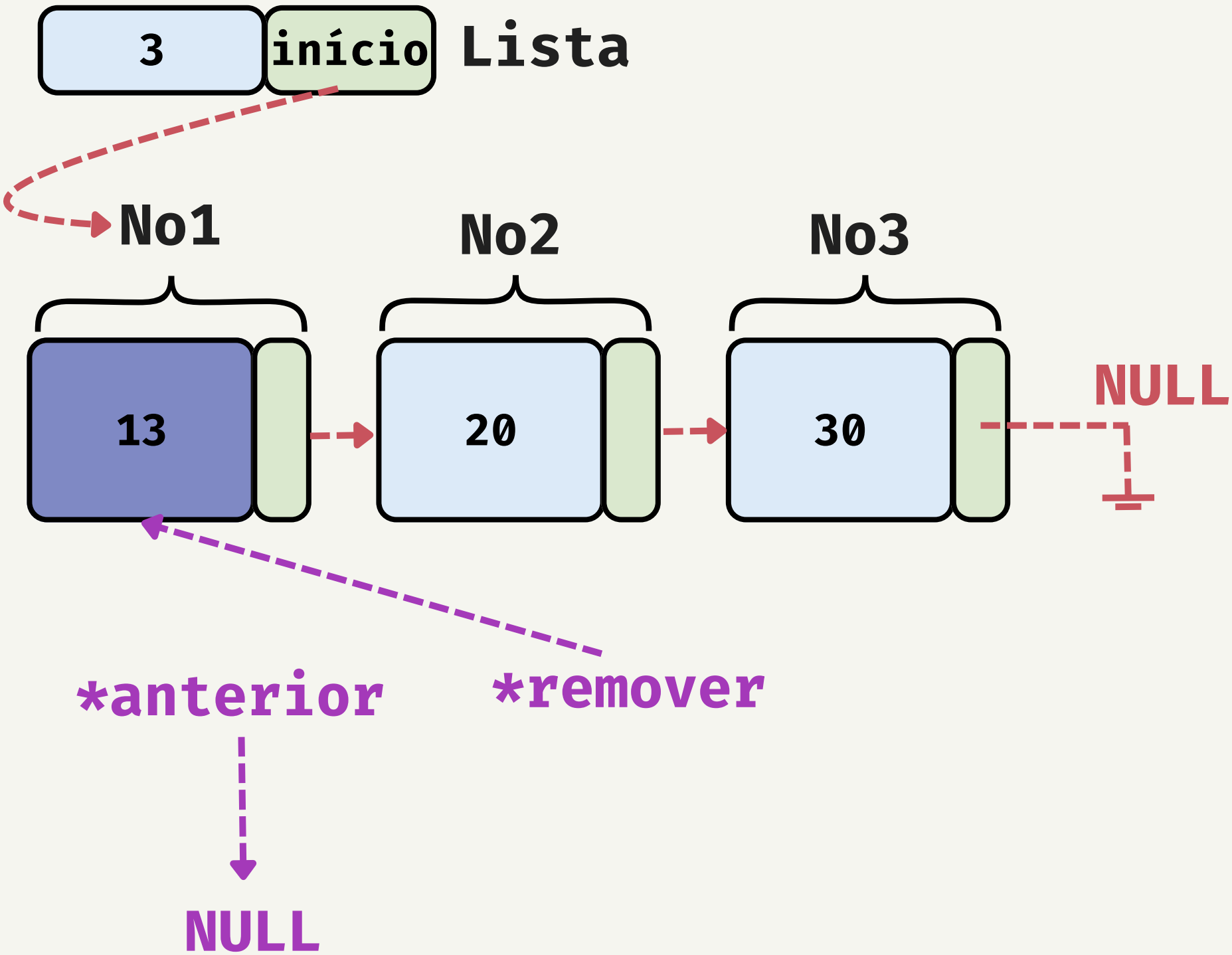
LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista
Remover valor = 13

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista

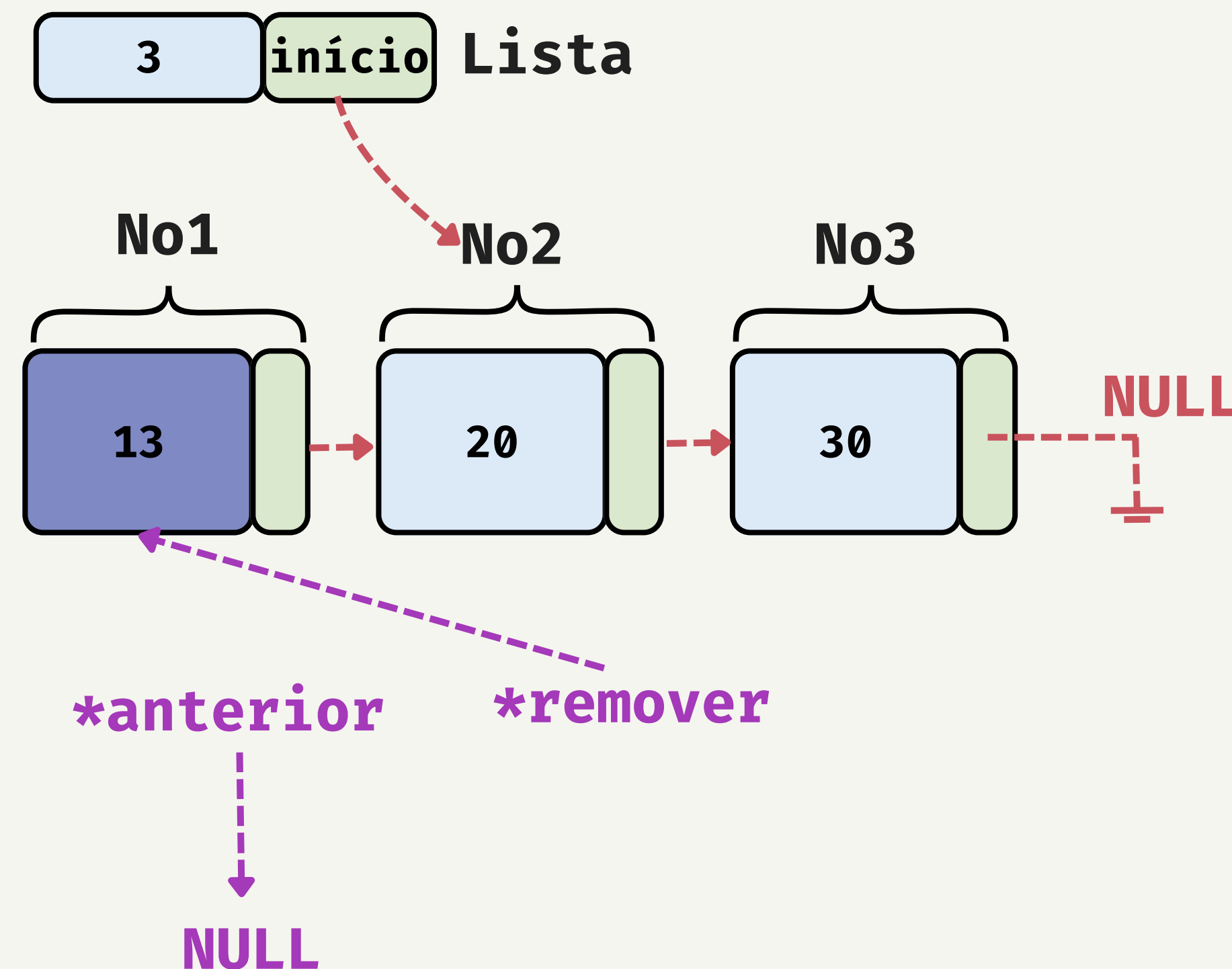
Remover valor = 13

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
    
```



LISTA ENCADEADA SIMPLES

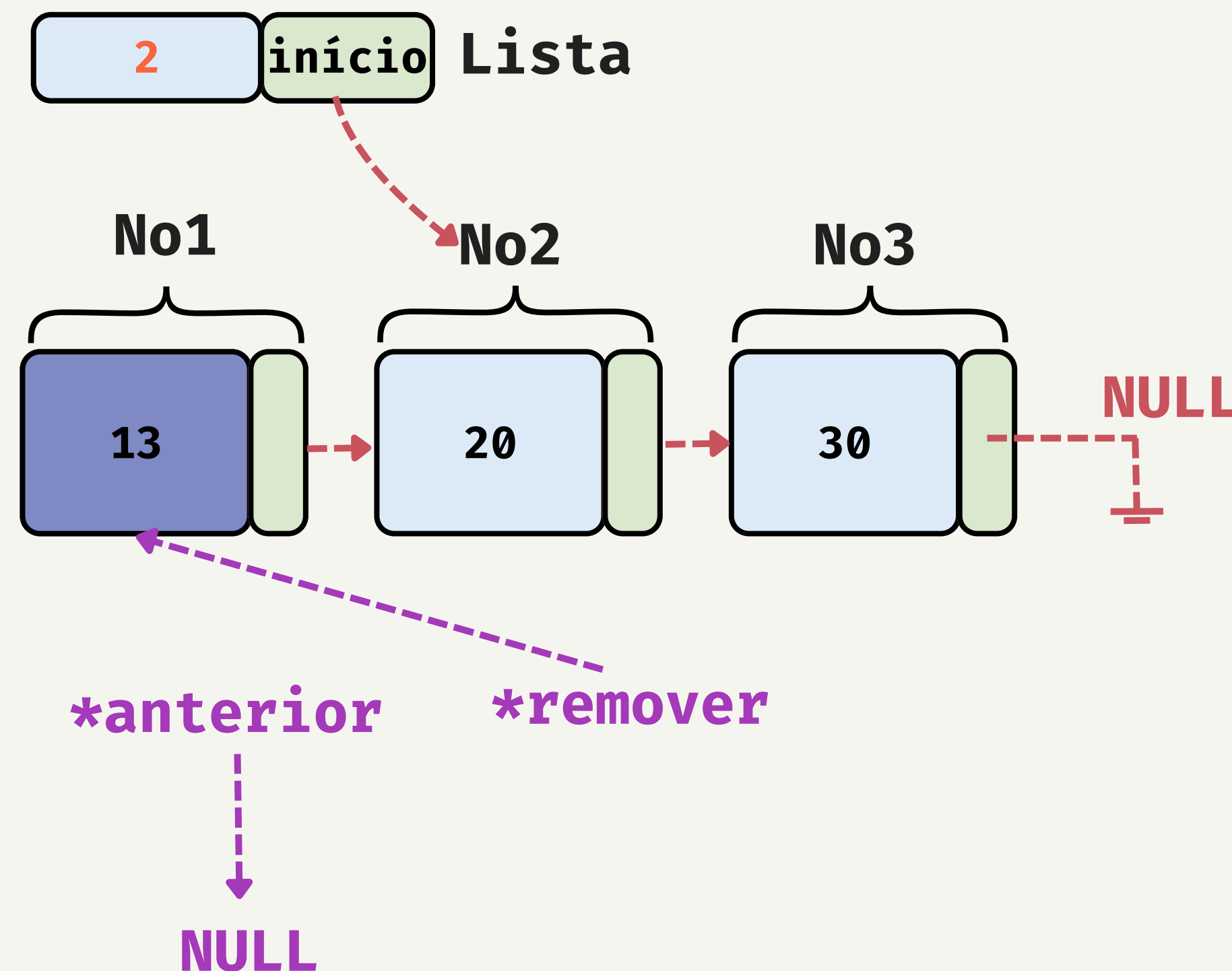
CASO 1: o nó é 1º da Lista
 Remover valor = 13

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
  
```



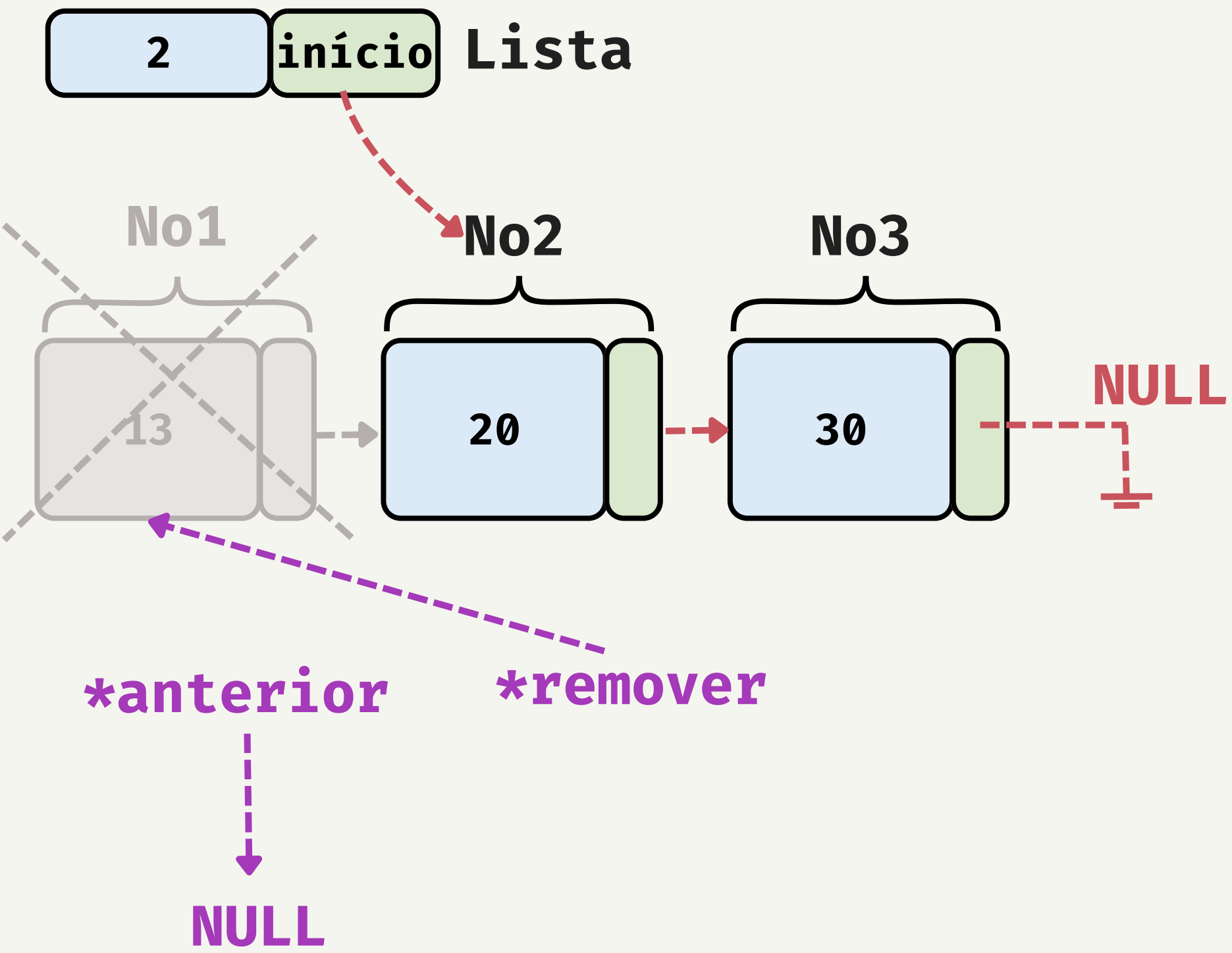
LISTA ENCADEADA SIMPLES

CASO 1: o nó é 1º da Lista
Remover valor = 13

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



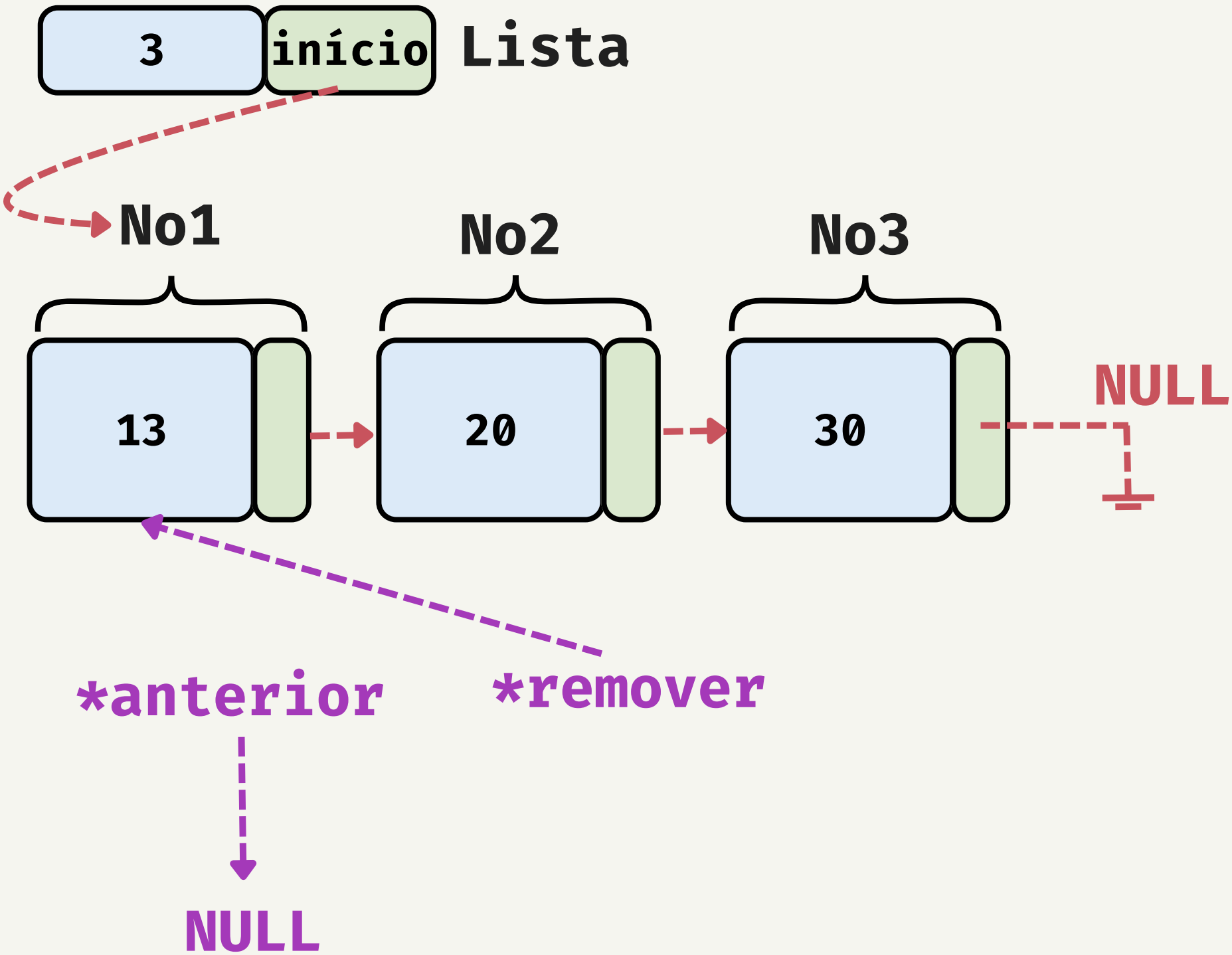
LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista
Remover valor = 20

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



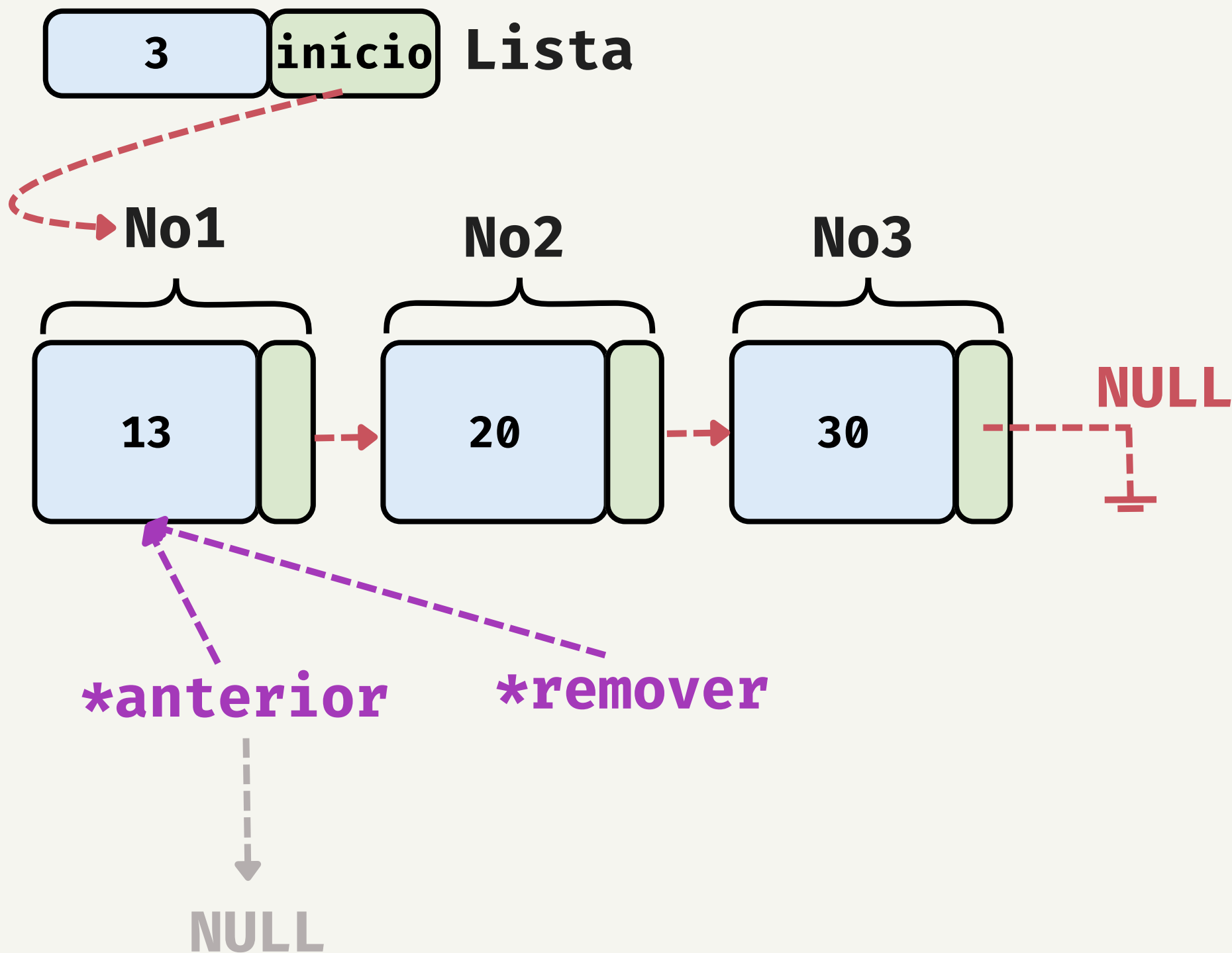
LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista
Remover valor = 20

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



LISTA ENCADEADA SIMPLES

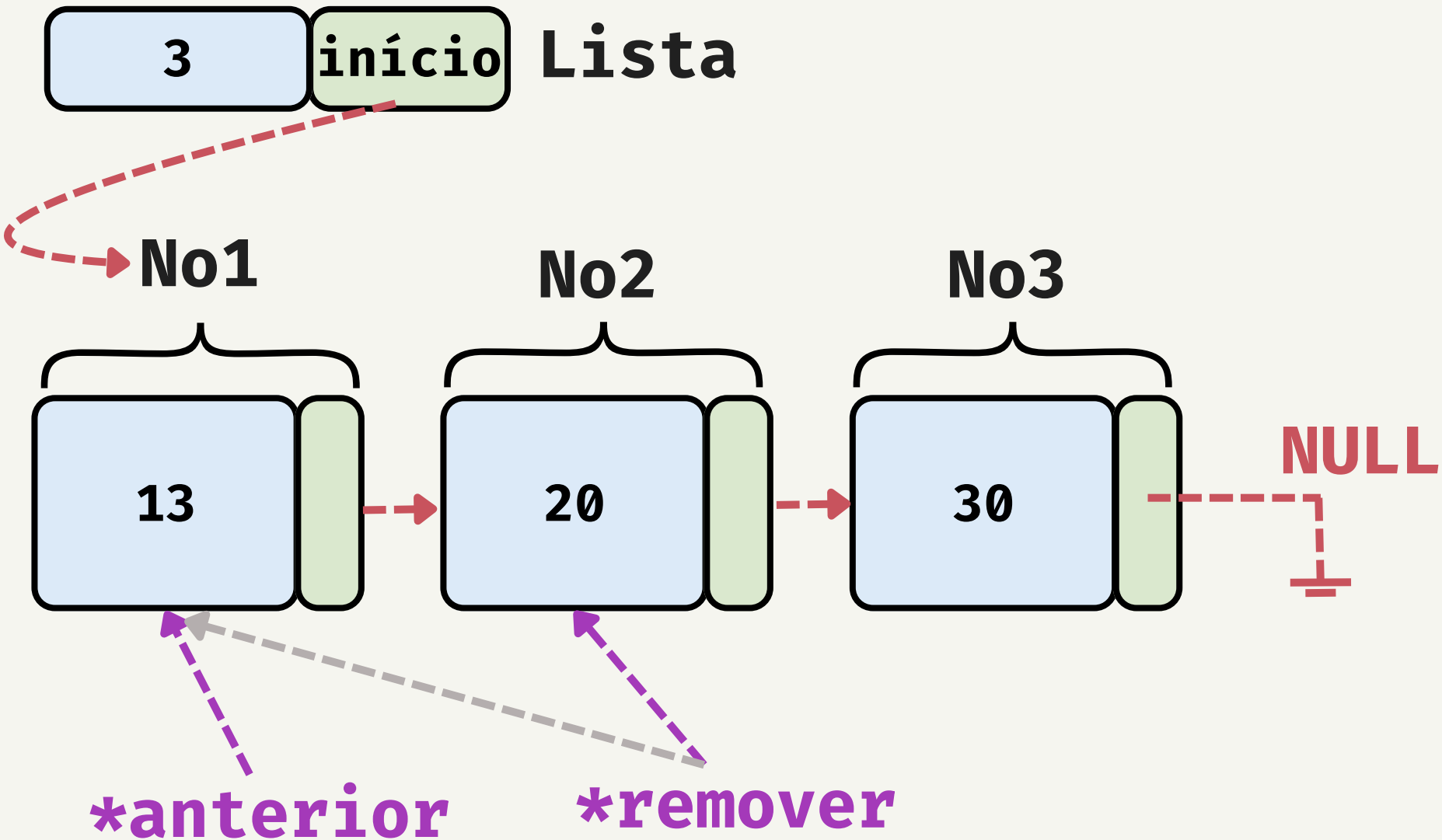
CASO 2: o nó está no meio da Lista
 Remover valor = 20

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
  
```



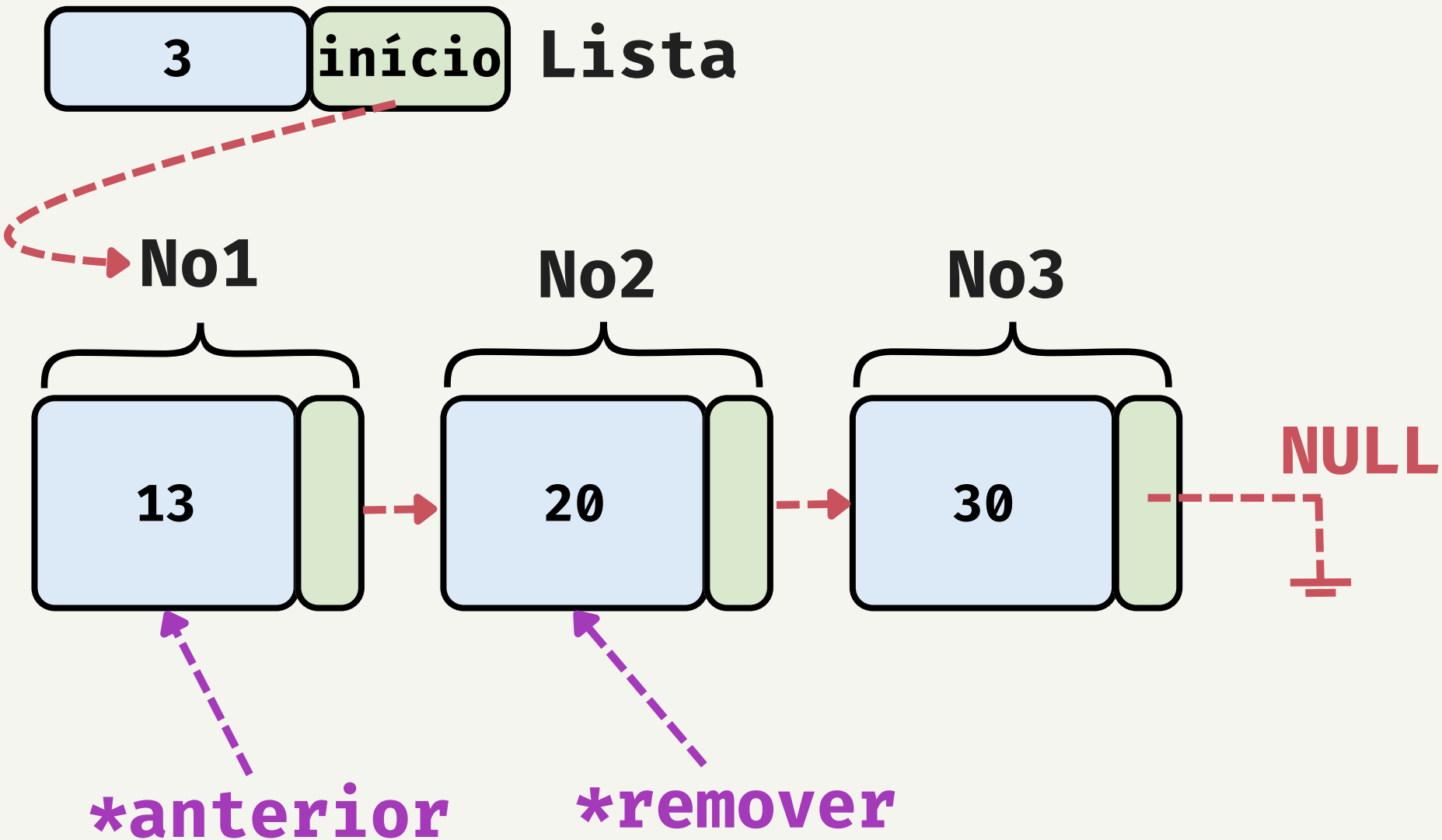
LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista
Remover valor = 20

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



LISTA ENCADEADA SIMPLES

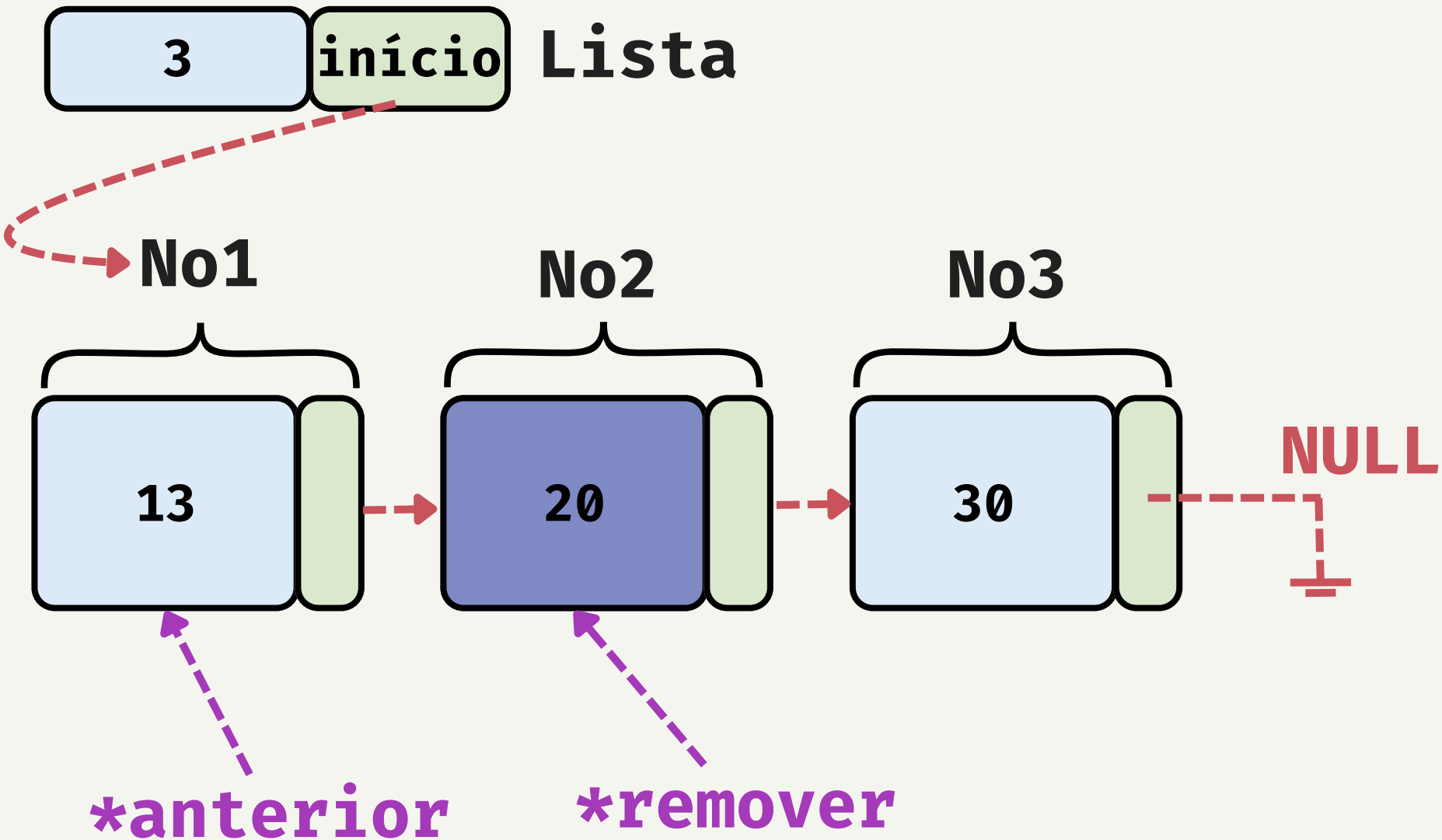
CASO 2: o nó está no meio da Lista
 Remover valor = 20

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
  
```



LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista

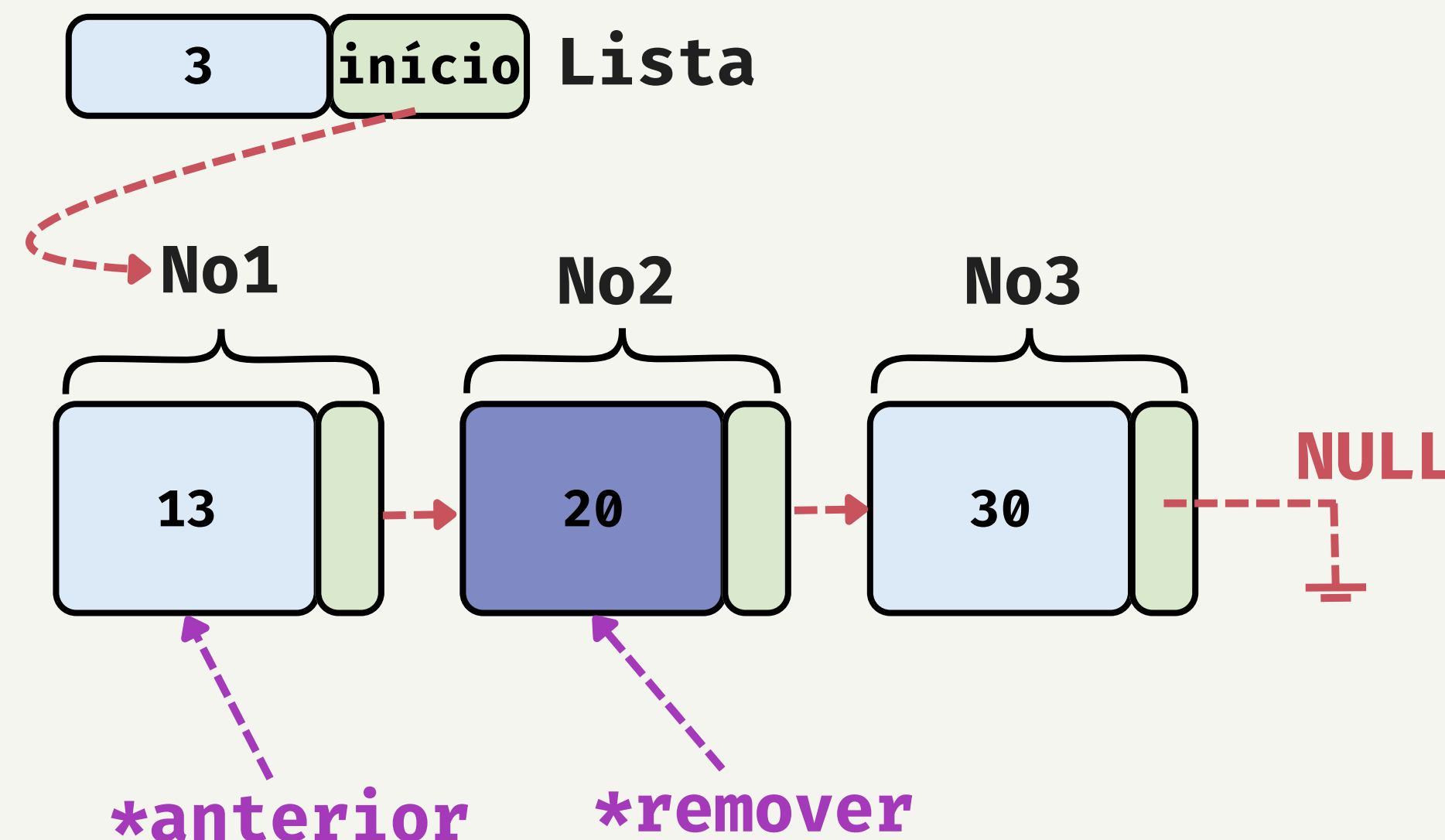
Remover valor = 20

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
    
```



0 nó não é o primeiro!

LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista

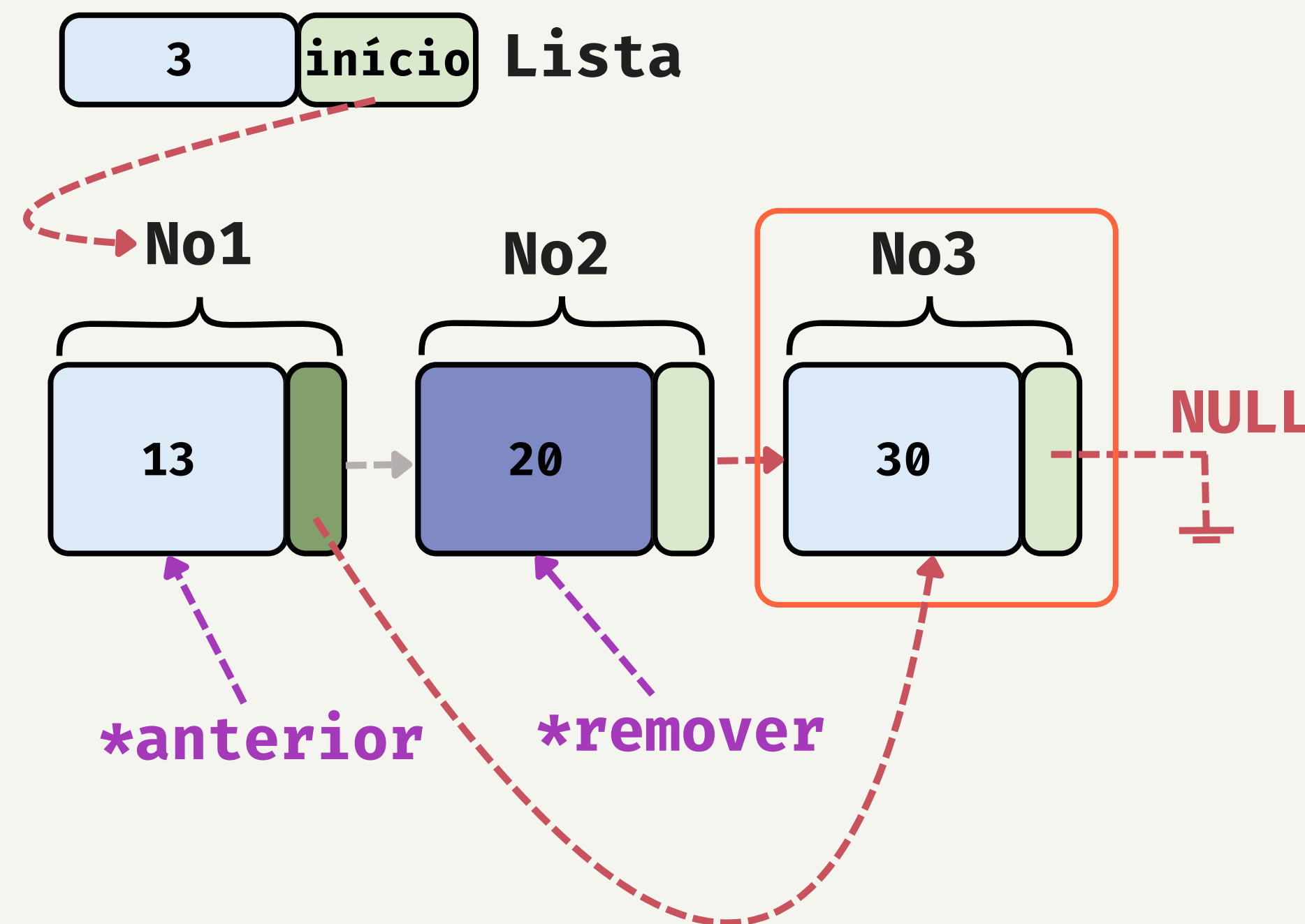
Remover valor = 20

```

No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
    
```



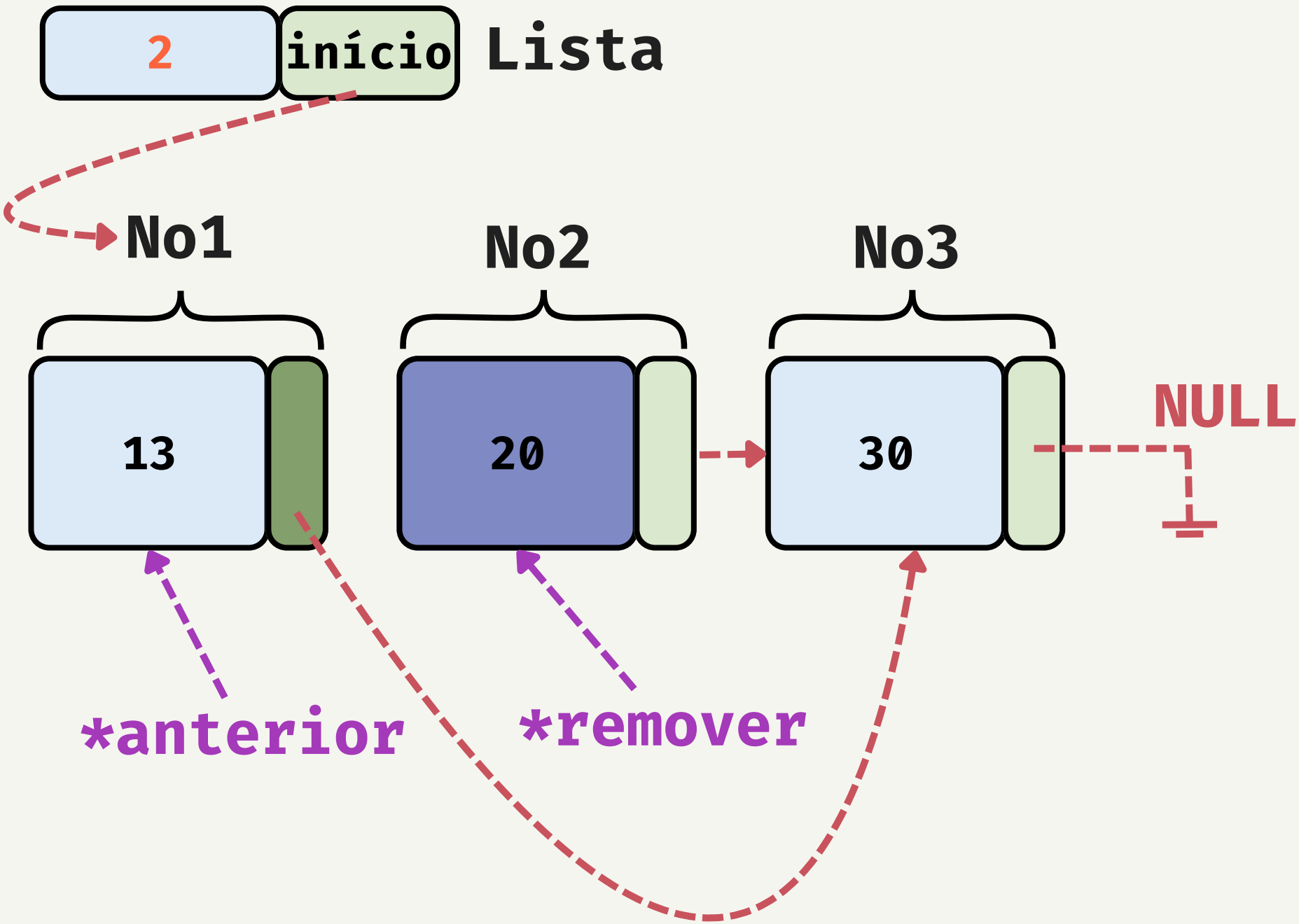
LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista
Remover valor = 20

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



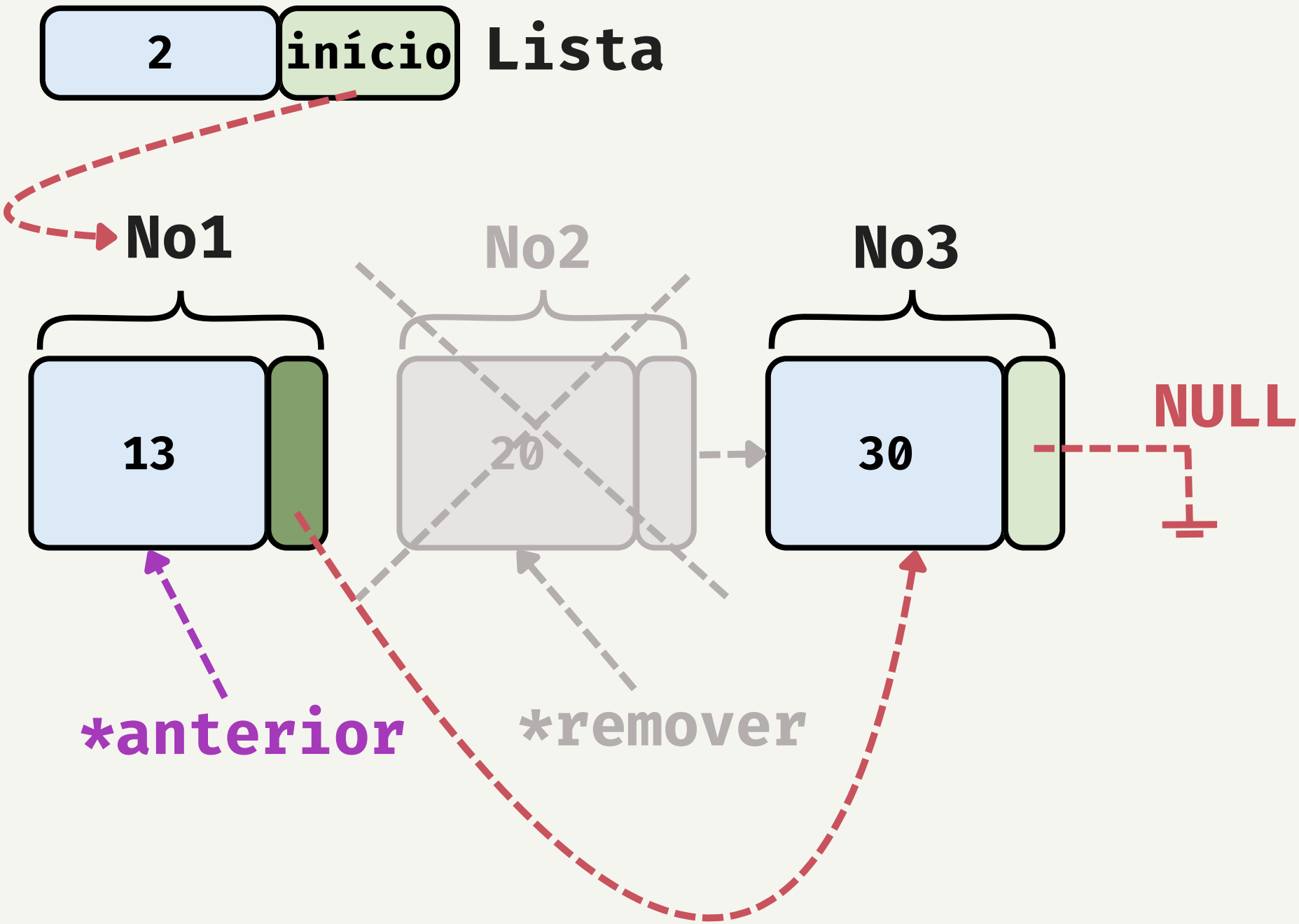
LISTA ENCADEADA SIMPLES

CASO 2: o nó está no meio da Lista
Remover valor = 20

```
No *remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```



LISTA ENCADEADA SIMPLES

Função **Remove()**

```
No *remove(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```

OBS: ***remove** aponta para um nó previamente alocado com **malloc**. Não estamos liberando a memória **free(remove)** do nó dentro da função. Isso desalocaria o ponteiro **remove** que eu quero retornar na função.

LISTA ENCADEADA SIMPLES

Função **Remove()**

```
No *remove(Lista* lista, int valor) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if(lista->inicio) {
        while(remover != NULL) {
            if(remover->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                lista->tamanho--;
                return remover;
            }
            anterior = remover;
            remover = remover->proxNo;
        }
    }

    return NULL;
}
```

OBS: ***remover** aponta para um nó previamente alocado com **malloc**. Não estamos liberando a memória **free(remover)** do nó dentro da função. Isso desalocaria o ponteiro **remover** que eu quero retornar na função.



Não esquecer de liberar o ponteiro quando for chamar a função no main():

```
No *removido = removeNo(lista, valor);
if(removido) {
    printf("Valor %d removido da lista.\n", valor);
    free(removido); // Libera a memória do nó removido
    removido = NULL; // Evita ponteiro pendente
} else {
    printf("Valor %d não encontrado para remoção.\n", valor);
}
```

LISTA ENCADEADA SIMPLES

Função **Remove()** sem retornar o nó

```
void remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* removerNo = lista->inicio;

    if(lista->inicio) {
        while(removerNo != NULL) {
            if(removerNo->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = removerNo->proxNo;
                } else {
                    anterior->proxNo = removerNo->proxNo;
                }

                free(removerNo); // Libera a memória do nó removido
                removerNo = NULL; // Evita ponteiro pendente
                lista->tamanho--;
                printf("Valor %d removido da lista.\n", valor);
                return; // Sai da função após remover o nó
            }
            anterior = removerNo;
            removerNo = removerNo->proxNo;
        }
    }

    printf("Valor %d não encontrado para remoção.\n", valor);
}
```

OBS: Apenas executa a ação de remover e libera o nó removido dentro da função.

no main():

```
printf("Digite o valor para remover: ");
scanf("%d", &valor);
remover(lista, valor);
```

LISTA ENCADEADA SIMPLES

Função **Imprimir()**

```
void imprimirLista(const Lista* lista) {  
    No* temp = lista->inicio;  
    while(temp) {  
        printf("%d -> ", temp->valorNo);  
        temp = temp->proxNo;  
    }  
    printf("NULL\n");  
}
```

LISTA ENCADEADA SIMPLES

Função **Imprimir()**

```
void imprimirLista(const Lista* lista) {  
    No* temp = lista->inicio;  
    while(temp) {  
        printf("%d -> ", temp->valorNo);  
        temp = temp->proxNo;  
    }  
    printf("NULL\n");  
}
```

OBS: const → Garante que a função não altere acidentalmente os dados da estrutura;

- **Útil** em funções que apenas **lê** os dados;

LISTA ENCADEADA SIMPLES

Função **Imprimir()**

```
void imprimirLista(const Lista* lista) {  
    No* temp = lista->inicio;  
    while(temp) {  
        printf("%d -> ", temp->valorNo);  
        temp = temp->proxNo;  
    }  
    printf("NULL\n");  
}
```

OBS: const → Garante que a função não altere acidentalmente os dados da estrutura;

- **Útil** em funções que apenas **lê** os dados;

Exemplo de saída:

1 -> 2 -> 3 -> 4 -> 5 -> NULL

LISTA ENCADEADA SIMPLES

Função **liberarLista()**

- evita vazamento de memória (**memory leak**)
- evita deixar nós alocados quando você terminou de usar a lista

```
void liberarLista(Lista* lista) {  
    No* atual = lista->inicio;  
    while (atual != NULL) {  
        No* temp = atual;  
        atual = atual->proxNo;  
        free(temp);  
    }  
    free(lista);  
}
```

LISTA ENCADEADA SIMPLES

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

Estratégia:

- **Percorrer a lista**
- **Atualizar lista** -> início se remover o início
- **Atualizar lista** -> tamanho
- Cuidar para **não** acessar memória já liberada

LISTA ENCADEADA SIMPLES

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

Estratégia:

- **Percorrer a lista**
- **Atualizar lista** -> início se remover o início
- **Atualizar lista** -> tamanho
- Cuidar para **não** acessar memória já liberada

Estrutura base:

```
typedef struct No {  
    int valorNo;  
    struct No* proxNo;  
} No;
```

```
typedef struct {  
    No* inicio;  
    int tamanho;  
} Lista;
```

LISTA ENCADEADA SIMPLES

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

Estratégia:

- **Percorrer a lista**
- **Atualizar lista** -> início se remover o início
- **Atualizar lista** -> tamanho
- Cuidar para **não** acessar memória já liberada

Estrutura base:

```
typedef struct No {  
    int valorNo;  
    struct No* proxNo;  
} No;
```

Mesma lógica do remover()

```
typedef struct {  
    No* inicio;  
    int tamanho;  
} Lista;
```

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

```
void remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* removerNo = lista->inicio;

    if(lista->inicio) {
        while(removerNo != NULL) {
            if(removerNo->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = removerNo->proxNo;
                } else {
                    anterior->proxNo = removerNo->proxNo;
                }
                free(removerNo);
                removerNo = NULL;
                lista->tamanho--;
                return;
            } else {
                anterior = removerNo;
                removerNo = removerNo->proxNo;
            }
        }
    }

    printf("Valor %d não encontrado para remoção.\n", valor);
}
```

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

```

void remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* removerNo = lista->inicio;

    if(lista->inicio) {
        while(removerNo != NULL) {
            if(removerNo->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = removerNo->proxNo;
                } else {
                    anterior->proxNo = removerNo->proxNo;
                }
                free(removerNo);
                removerNo = NULL;
                lista->tamanho--;
                return;
            } else {
                anterior = removerNo;
                removerNo = removerNo->proxNo;
            }
        }

        printf("Valor %d não encontrado para remoção.\n", valor);
    }
}
    
```

```

void removerPares(Lista* lista) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if (lista->inicio) {
        while (remover != NULL) {
            if (remover->valorNo % 2 == 0) {
                if (anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                No* temp = remover;
                remover = remover->proxNo;
                free(temp);
                temp = NULL;
                lista->tamanho--;
            } else {
                anterior = remover;
                remover = remover->proxNo;
            }
        }
    }
}
    
```

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

```

void remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* removerNo = lista->inicio;

    if(lista->inicio) {
        while(removerNo != NULL) {
            if(removerNo->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = removerNo->proxNo;
                } else {
                    anterior->proxNo = removerNo->proxNo;
                }
                free(removerNo);
                removerNo = NULL;
                lista->tamanho--;
                return;
            } else {
                anterior = removerNo;
                removerNo = removerNo->proxNo;
            }
        }

        printf("Valor %d não encontrado para remoção.\n", valor);
    }
}
    
```

```

void removerPares(Lista* lista) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if (lista->inicio) {
        while (remover != NULL) {
            if (remover->valorNo % 2 == 0) {
                if (anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                No* temp = remover;
                remover = remover->proxNo;
                free(temp);
                temp = NULL;
                lista->tamanho--;
            } else {
                anterior = remover;
                remover = remover->proxNo;
            }
        }
    }
}
    
```

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

```

void remover(Lista* lista, int valor) {
    No* anterior = NULL;
    No* removerNo = lista->inicio;

    if(lista->inicio) {
        while(removerNo != NULL) {
            if(removerNo->valorNo == valor) {
                if(anterior == NULL) {
                    lista->inicio = removerNo->proxNo;
                } else {
                    anterior->proxNo = removerNo->proxNo;
                }
                free(removerNo);
                removerNo = NULL;
                lista->tamanho--;
                return;
            } else {
                anterior = removerNo;
                removerNo = removerNo->proxNo;
            }
        }

        printf("Valor %d não encontrado para remoção.\n", valor);
    }
}
    
```

```

void removerPares(Lista* lista) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if (lista->inicio) {
        while (remover != NULL) {
            if (remover->valorNo % 2 == 0) {
                if (anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }
                No* temp = remover;
                remover = remover->proxNo;
                free(temp);
                temp = NULL;
                lista->tamanho--;
            } else {
                anterior = remover;
                remover = remover->proxNo;
            }
        }
    }
}
    
```

SEMPRE GUARDE O NÓ QUE SERÁ REMOVIDO ANTES DE AVANÇAR O PONTEIRO.

Exemplo: Dada uma lista encadeada simples de inteiros, remover **todos os pares**.

Use **ponteiro temporário** quando uma **atualização** (ou **free**) pode te fazer **perder o endereço do nó** que ainda precisa acessar.

Sempre guarde o nó que será removido antes de avançar o ponteiro.

- Guarda** o nó que será **removido** (temp = remover)
- Avança** o ponteiro de caminhada **antes** do free (remover = remover->proxNo)
- Libera** o nó **guardado** (free(temp))

```

void removerPares(Lista* lista) {
    No* anterior = NULL;
    No* remover = lista->inicio;

    if (lista->inicio) {
        while (remover != NULL) {
            if (remover->valorNo % 2 == 0) {
                if (anterior == NULL) {
                    lista->inicio = remover->proxNo;
                } else {
                    anterior->proxNo = remover->proxNo;
                }

                No* temp = remover;
                remover = remover->proxNo;
                free(temp);
                temp = NULL;
                lista->tamanho--;
            } else {
                anterior = remover;
                remover = remover->proxNo;
            }
        }
    }
}
    
```

Exercício casa: Catálogo de Livros

Estrutura base:

```
typedef struct No {  
    int codigo;  
    float preco;  
    struct No* proxNo;  
} No;
```

```
typedef struct {  
    No* inicio;  
    int tamanho;  
} Lista;
```

Implemente funções que:

1. Remova todos os livros cujo preço seja maior que limite.
2. Retorne o ponteiro para o nó com maior preço.
3. Remova nós duplicados por código, mantendo a primeira ocorrência.
Considere que dois livros são duplicados se tiverem o mesmo código.